



**Институт
интеллектуальных кибернетических систем
Кафедра №22 «Кибернетика»**

Направление подготовки 01.03.02 Прикладная математика и информатика

Отчет

о производственной практике на тему:

Разработка информационно-аналитической системы для моделирования и
прогнозирования динамики биржевых котировок

Группа	Б22-511		Рябов Н.А.
Студент	_____	_____	_____
	(подпись)		(ФИО)
Руководитель	_____	_____	Букалин А.О.
	(подпись)		(ФИО)
Научный консультант	_____	_____	_____
	(подпись)		(ФИО)
Оценка руководителя	_____	Оценка консультанта	_____
	(0-100 баллов)		(0-100 баллов)
Итоговая оценка	_____	ECTS	_____
	(0-100 баллов)		
Комиссия			
Председатель	_____	_____	_____
	(подпись)		(ФИО)
	_____	_____	_____
	(подпись)		(ФИО)
	_____	_____	_____
	(подпись)		(ФИО)
	_____	_____	_____
	(подпись)		(ФИО)

Москва 2026

Реферат

Пояснительная записка содержит 4 главы, в которых рассматриваются основные аспекты разработки информационно-аналитической системы для моделирования и прогнозирования динамики биржевых котировок.

Объем работы составляет 34 страниц, с учетом приложений — 66. Количество приложений — 10, количество использованных источников — 37.

Ключевые слова: биржевые котировки, временные ряды, прогнозирование, Python, информационно-аналитическая система, React.js, визуализация, финансовые данные, алгоритмы прогноза, нейронные сети.

Целью работы является разработка информационно-аналитической системы для прогнозирования динамики биржевых котировок на основе современных методов анализа временных рядов.

В первой главе проводится обзор существующих методов прогнозирования временных рядов, включая статистические, нейросетевые и комбинированные подходы.

Во второй главе приводится математическое описание моделей прогнозирования (классических, нейросетевых и ансамблевых), алгоритмов обработки данных и построения прогноза для информационно-аналитической системы.

Третья глава посвящена проектированию архитектуры информационно-аналитической системы: определены требования, разработана модульная структура серверной части на FastAPI и спроектирован пользовательский веб-интерфейс с организацией взаимодействия компонентов через REST API.

Четвертая глава посвящена программной реализации информационно-аналитической системы: описаны выбранные технологии, структура модулей загрузки данных, статистических и нейросетевых алгоритмов, пользовательского интерфейса

Содержание

Введение	4
Глава 1. Аналитический обзор и методы прогнозирования биржевых котировок	6
1.1 Биржевые котировки и их роль в финансовых рынках	6
1.2 Биржевые котировки как временные ряды	6
1.3 Основные подходы к прогнозированию биржевых котировок	7
1.4 Статистические методы прогнозирования временных рядов	7
1.5 Использование программных средств для прогнозирования	8
1.6 Нейросетевые методы прогнозирования временных рядов	8
1.7 Обзор существующих систем прогнозирования биржевых котировок	10
1.8 Выводы	11
1.9 Цели и задачи ВКР	12
Глава 2. Выбор моделей и алгоритмов для прогнозирования биржевых котировок	13
2.1 Обоснование выбора математического аппарата	13
2.2 Математические модели прогнозирования	13
2.3 Алгоритм обработки данных	14
2.4 Алгоритм построения прогноза	16
2.5 Выводы	18
Глава 3. Проектирование архитектуры информационно-аналитической системы прогнозирования биржевых котировок	19
3.1 Функциональные и нефункциональные требования к системе	19
3.2 Проектирование архитектуры серверной части	20
3.3 Проектирование программного интерфейса системы	21
3.4 Организация взаимодействия между компонентами системы	22
3.5 Выводы	23
Глава 4. Программная реализация информационно-аналитической системы	23
4.1 Выбор инструментов и технологий разработки	23
4.2 Реализация модуля загрузки и обработки данных на Python	24
4.3 Реализация статистических алгоритмов	25
4.4 Реализация нейросетевых моделей	27
4.5 Реализация пользовательского интерфейса	28
4.6 Проверка работы системы	32
4.7 Выводы	32
Заключение	33
Список литературы	35
А. Приложение А. Листинг программной реализации для прогноза котировок	37
Листинг А.1 – Python код для загрузки и обработки исторических данных	37
Листинг А.2 – Python код модели наивного прогноза	39
Листинг А.3 – Python код модели скользящего среднего	40
Листинг А.4 – Python код модели экспоненциального сглаживания	41
Листинг А.5 – Python код модели ARIMA	42
Листинг А.6 – Python код модели Prophet	43
Листинг А.7 – Python код модели LSTM	44
Листинг А.8 – Python код модели GRU	46
Листинг А.9 – Python код модели transformer	47
Листинг А.10 – JavaScript-код веб приложения	50

Введение

В условиях развития рыночной экономики анализ финансовых данных приобретает всё большее значение. Биржевые котировки являются одним из ключевых источников информации о состоянии финансовых рынков и экономических процессов. Их динамика отражает изменения спроса и предложения, реакцию рынка на внешние события, а также ожидания участников торгов. В связи с этим задача анализа и прогнозирования биржевых котировок остаётся актуальной как для профессиональных участников рынка, так и для исследователей в области прикладной математики и информационных технологий.

Биржевые котировки представляют собой временные ряды, характеризующиеся сложной структурой, наличием случайных колебаний и зависимостей от предыдущих значений. Прогнозирование таких данных является нетривиальной задачей, требующей применения специальных методов математического и статистического анализа. При этом ручная обработка и анализ больших объёмов данных являются трудоёмкими и неэффективными, что обуславливает необходимость использования автоматизированных информационно-аналитических систем.

Информационно-аналитические системы позволяют объединить процессы сбора данных, их обработки, анализа и представления результатов в едином программном комплексе. Применение таких систем в финансовой сфере способствует повышению наглядности анализа, сокращению времени на расчёты и снижению вероятности ошибок, связанных с человеческим фактором. Особенно актуальной является разработка систем, ориентированных на моделирование и прогнозирование временных рядов биржевых котировок.

Современные средства программирования предоставляют широкие возможности для создания подобных систем. Язык программирования Python широко используется для анализа данных и работы с временными рядами благодаря наличию специализированных библиотек для численных вычислений, статистического анализа и визуализации. В то же время веб-технологии позволяют реализовывать пользовательские интерфейсы, обеспечивающие удобный доступ к результатам анализа и их наглядное представление. Использование клиентской части на основе React.js делает возможной интерактивную демонстрацию прогнозных данных в виде графиков.

Актуальность данной работы заключается в необходимости разработки наглядного и доступного программного решения для анализа и прогнозирования динамики биржевых котировок, объединяющего вычислительную часть и средства визуализации. Такая система может быть использована как в учебном процессе, так и для демонстрации принципов прогнозирования временных рядов.

В первой главе проведён анализ предметной области, рассмотрены биржевые котировки как временные ряды и их основные свойства. Выполнен обзор классических статистических, нейросетевых и комбинированных методов прогнозирования, а также программных средств их реализации, что послужило теоретической основой для дальнейшей разработки системы.

Во второй главе приведено описание математического аппарата для прогнозирования биржевых котировок, включающего классические статистические модели (ARIMA, Prophet и др.), нейросетевые архитектуры (LSTM, GRU, Transformer) и ансамблевый подход. Приведены математические формализации выбранных моделей, а также описаны алгоритмы обработки данных и построения прогноза, образующие теоретическую базу для программной реализации системы.

В третьей главе сформулированы функциональные и нефункциональные требования к системе и на их основе спроектирована модульная архитектура серверной части на базе фреймворка FastAPI с разделением на слабо связанные подсистемы. Разработана структура клиентского веб-интерфейса в виде одностраничного приложения, описаны его основные компоненты и принципы взаимодействия между клиентом и сервером по HTTP с обменом данными в формате JSON.

В четвёртой главе описана практическая реализация разработанной системы: обоснован выбор инструментов и технологий (Python, FastAPI, PyTorch, React), представлена структура программного кода модулей загрузки данных, статистических и нейросетевых алгоритмов, а также пользовательского веб-интерфейса. Приведены результаты экспериментальной проверки системы на реальных биржевых данных, подтверждающие работоспособность и сопоставимую эффективность классических, нейросетевых и ансамблевых методов прогнозирования.

Глава 1. Аналитический обзор и методы прогнозирования биржевых котировок

В данной главе проводится аналитический обзор предметной области, связанной с моделированием и прогнозированием динамики биржевых котировок. Рассматриваются основные понятия, используемые при анализе финансовых рынков, а также особенности биржевых котировок как временных рядов.

В рамках главы анализируются характерные свойства временных рядов, такие как изменчивость, наличие случайных колебаний и зависимость текущих значений от предыдущих. Особое внимание уделяется обзору существующих подходов и методов прогнозирования биржевых котировок, включая классические статистические методы и программные средства их реализации.

Результаты аналитического обзора, представленные в данной главе, служат теоретической основой для дальнейшей разработки модели информационно-аналитической системы и обоснования выбора методов прогнозирования, используемых в последующих главах работы.

1.1 Биржевые котировки и их роль в финансовых рынках

Биржевые котировки представляют собой числовые значения, отражающие текущую цену финансового актива на бирже в определённый момент времени. В качестве таких активов могут выступать акции, облигации, валютные пары, товары и производные финансовые инструменты. Котировки формируются в результате взаимодействия спроса и предложения и являются одним из основных источников информации о состоянии финансового рынка.[1]

Динамика биржевых котировок зависит от множества факторов, включая экономические показатели, финансовые отчёты компаний, политические события, изменения процентных ставок и общее состояние мировой экономики. Кроме того, на котировки влияет поведение участников рынка, их ожидания и реакции на новости. В результате этого ценовые изменения носят сложный и зачастую нелинейный характер.

Анализ биржевых котировок используется для решения различных задач, таких как оценка текущей рыночной ситуации, выявление тенденций изменения цен, управление инвестиционными рисками и принятие решений о покупке или продаже финансовых активов. Одной из наиболее сложных и в то же время востребованных задач является прогнозирование будущих значений котировок на основе имеющихся данных.

1.2 Биржевые котировки как временные ряды

С точки зрения математического и статистического анализа биржевые котировки

представляют собой временные ряды [2], то есть последовательности значений, упорядоченных по времени. Каждое значение временного ряда соответствует цене финансового актива в конкретный момент или интервал времени, например, за день, час или минуту.

Финансовые временные ряды обладают рядом характерных особенностей. Во-первых, им свойственна высокая изменчивость, связанная с быстрыми колебаниями цен. Во-вторых, такие ряды часто содержат шум, затрудняющий выявление устойчивых закономерностей. В-третьих, временные ряды биржевых котировок могут демонстрировать наличие трендов, сезонных компонентов и циклических колебаний.

Кроме того, для финансовых временных рядов характерна зависимость текущих значений от предыдущих [3], что делает возможным применение методов прогнозирования, основанных на анализе прошлой динамики. Вместе с тем наличие случайных факторов и внешних воздействий существенно ограничивает точность прогнозов и требует осторожного подхода к интерпретации результатов.

1.3 Основные подходы к прогнозированию биржевых котировок

Методы прогнозирования биржевых котировок можно условно разделить на несколько основных групп в зависимости от используемых математических и алгоритмических подходов.

Одним из традиционных направлений является применение статистических методов анализа временных рядов. [4] Такие методы основаны на выявлении закономерностей в исторических данных и построении математических моделей, описывающих поведение временного ряда. Преимуществом статистических методов является их относительная простота и интерпретируемость, однако они часто предполагают выполнение определённых предположений о структуре данных, что не всегда соответствует реальным финансовым процессам.

Другую группу составляют методы, основанные на использовании алгоритмов машинного обучения. [5] Эти подходы позволяют учитывать сложные зависимости в данных и не требуют строгих предположений о виде модели. Вместе с тем такие методы могут быть чувствительны к качеству исходных данных и требуют значительных вычислительных ресурсов и аккуратной настройки параметров.

Также используются комбинированные подходы, сочетающие элементы статистического анализа и алгоритмических методов. Они позволяют повысить качество прогнозирования за счёт объединения преимуществ различных подходов, однако усложняют процесс разработки и интерпретации моделей.

1.4 Статистические методы прогнозирования временных рядов

Классические статистические методы прогнозирования временных рядов основаны

на анализе структуры данных и использовании прошлых значений ряда для оценки будущих. К таким методам относятся скользящие средние, экспоненциальное сглаживание, а также авторегрессионные модели.

Методы скользящих средних применяются для сглаживания временного ряда и выявления общей тенденции изменения котировок. Они просты в реализации и позволяют уменьшить влияние краткосрочных колебаний, однако не подходят для точного краткосрочного прогнозирования.

Простое скользящее среднее представляет собой сумму цен закрытия (P_i) за N периодов, разделенная на N .

Экспоненциальное сглаживание учитывает прошлые значения временного ряда с различными весами, придавая больший вес более свежим данным.

Такие методы широко применяются для прогнозирования временных рядов с выраженным трендом или сезонностью.

Авторегрессионные модели предполагают, что текущее значение временного ряда зависит от нескольких предыдущих значений. Эти модели позволяют учитывать внутреннюю структуру временного ряда, однако их применение требует предварительного анализа данных и проверки выполнения статистических предположений.

1.5 Использование программных средств для прогнозирования

Современные задачи прогнозирования биржевых котировок, как правило, решаются с использованием программных средств, позволяющих автоматизировать обработку данных и расчёт прогнозов. Программная реализация делает возможным работу с большими объёмами данных, проведение серии экспериментов и сравнительный анализ различных методов.

Язык программирования Python получил широкое распространение в области анализа данных благодаря своей простоте и наличию большого количества библиотек для работы с временными рядами и численными методами [6]. Он позволяет реализовывать как классические статистические методы, так и более сложные алгоритмы прогнозирования.

Дополнительным преимуществом программных решений является возможность визуализации результатов прогнозирования. Графическое представление временных рядов и прогнозных значений способствует более наглядному анализу данных и облегчает интерпретацию полученных результатов, особенно при использовании веб-интерфейсов.

1.6 Нейросетевые методы прогнозирования временных рядов

Нейросетевые подходы занимают особое место в задачах прогнозирования финансовых временных рядов благодаря своей способности аппроксимировать сложные нелинейные зависимости без предварительного задания строгой параметрической модели. В отличие от классических статистических методов, нейронные сети не требуют

выполнения предположений о стационарности, нормальности распределения остатков или форме функциональной связи между прошлыми и будущими значениями ряда. Основной идеей является обучение многослойной структуры отображению последовательности прошлых наблюдений в будущее значение (или последовательность значений) котировки.

Наиболее простой нейросетевой архитектурой для прогнозирования временных рядов является многослойный персептрон. [7] В такой постановке временной ряд преобразуется в множество примеров «вход–выход», где на вход сети подаётся вектор лаговых значений цены (например, котировки закрытия за P предыдущих периодов), а целевым выходом служит прогнозируемое значение.

Последовательное соединение нескольких скрытых слоёв позволяет аппроксимировать весьма сложные функции, однако многослойный персептрон не учитывает естественный порядок следования наблюдений во времени, рассматривая каждый входной вектор независимо.

Теоретически рекуррентные сети способны запоминать зависимости на длинных интервалах времени, однако на практике при обучении классических RNN возникает проблема затухающих или взрывающихся градиентов, затрудняющая захват долгосрочных закономерностей в динамике биржевых котировок.

Для преодоления указанного ограничения были разработаны сети с долгой краткосрочной памятью (LSTM), [8] а также их упрощённый вариант — управляемые рекуррентные блоки (GRU). Ключевым элементом LSTM является ячейка памяти, состояние которой регулируется тремя вентилями: вентилем забывания, входным и выходным.

Благодаря такой структуре LSTM способны длительное время сохранять значимую информацию, что делает их широко распространёнными при моделировании динамики биржевых котировок, характеризующейся сложными нелинейными трендами, сезонными эффектами и периодами высокой волатильности.

В последние годы значительное внимание при прогнозировании финансовых рядов уделяется механизмам внимания и архитектуре трансформера. Модели на основе механизма Self-Attention позволяют сопоставлять каждый элемент последовательности со всеми остальными, вычисляя веса внимания и формируя контекстное представление [9], адаптивно выделяющее наиболее влиятельные временные точки. Такой подход позволяет эффективно улавливать как краткосрочные, так и дальние зависимости в динамике цен без ограничений, свойственных рекуррентным архитектурам.

Применение нейросетевых методов прогнозирования биржевых котировок сопряжено с рядом практических аспектов. С одной стороны, нейронные сети способны улавливать сложные нелинейные связи и взаимодействия между различными факторами

без предварительного подбора параметрической модели. С другой стороны, данные методы предъявляют повышенные требования к объёму и качеству обучающей выборки, чувствительны к настройке гиперпараметров и склонны к переобучению при недостаточной регуляризации. Кроме того, глубокие модели часто рассматриваются как «чёрные ящики», что затрудняет содержательную интерпретацию полученных прогнозов. Тем не менее, в рамках разработки информационно-аналитической системы нейросетевые решения рассматриваются как гибкий и мощный инструмент, способный дополнить классические статистические подходы, обеспечивая более высокую точность при условии тщательной предобработки данных и обоснованного выбора архитектуры сети.

1.7 Обзор существующих систем прогнозирования биржевых котировок

Инструменты прогнозирования финансовых временных рядов, доступные сегодня, можно условно разделить на три группы: специализированные библиотеки для языков программирования, встроенные средства торговых терминалов и отдельные веб-ориентированные сервисы. Каждая из этих групп имеет свою область применения и характерные ограничения.

Наиболее гибким инструментарием обладают статистические пакеты и библиотеки машинного обучения, ориентированные на пользователей с навыками программирования.

В среде Python разработчиков широкое распространение получила библиотека `statsmodels`, [10] предоставляющая реализации классических моделей – от экспоненциального сглаживания до ARIMA с автоматическим подбором порядков. Рядом с ней существует библиотека `pmddarima`, реализующая алгоритм пошагового перебора параметров ARIMA-моделей. [11] Для байесовского структурного моделирования временных рядов применяется библиотека `Prophet`, позволяющая учитывать тренд, несколько сезонов и эффект праздничных дней. [12]

Глубокие нейронные сети для прогнозирования реализуются средствами `PyTorch` и `Keras`. [13] Эти фреймворки предоставляют низкоуровневые строительные блоки – слои, функции потерь, оптимизаторы, – но всю архитектуру модели, цикл обучения и предобработку данных пользователь проектирует самостоятельно. Существуют и более специализированные надстройки, такие как `pytorch-forecasting` и `darts`, которые предлагают готовые реализации LSTM, GRU и Transformer для временных рядов.

Основной недостаток всех перечисленных библиотек – необходимость написания программного кода. Пользователь должен самостоятельно загружать данные, разбивать их, конфигурировать модель, обучать её и визуализировать результат. Для аналитика или трейдера, не владеющего программированием на достаточном уровне, этот путь практически недоступен. Кроме того, сравнение нескольких моделей требует отдельного написания скриптов и ручного сведения результатов.

Торговые платформы, такие как MetaTrader [14], TradingView [15] и Thinkorswim [16], ориентированы на практикующих трейдеров и предоставляют встроенные индикаторы технического анализа. В их числе – скользящие средние, полосы Боллинджера, MACD и ряд других. Пользователь может накладывать эти индикаторы на график цены и на их основе строить субъективные суждения о будущем движении.

Некоторые терминалы позволяют создавать собственные индикаторы на встроенных скриптовых языках (MQL в MetaTrader, Pine Script в TradingView). Это расширяет аналитические возможности, но не меняет сути: терминалы остаются инструментами визуализации и ручного анализа, а не автоматического прогнозирования.

Главное ограничение таких платформ – отсутствие полноценных прогнозных моделей. Пользователь видит сглаженные значения или осцилляторы, но не получает количественного прогноза на заданный горизонт с оценкой точности. Нейросетевые методы, ансамбли и автоматический подбор параметров в терминалах не представлены вовсе. Метрики качества прогноза не рассчитываются, что лишает пользователя возможности объективно сравнивать подходы.

Отдельную нишу занимают веб-сервисы, предлагающие прогнозирование без написания кода. Сюда можно отнести облачные платформы вроде Google Vertex AI [17], Amazon Forecast [18] и DataRobot [19]. Они предоставляют графический интерфейс для загрузки данных и выбора модели, а обучение выполняют на серверных мощностях провайдера. Amazon Forecast, в частности, автоматически перебирает несколько статистических и нейросетевых алгоритмов и выбирает лучший [20].

Однако подобные сервисы ориентированы в первую очередь на задачи бизнес-прогнозирования (спрос, продажи, запасы) и плохо адаптированы к специфике биржевых котировок – высокой волатильности, нестационарности и отсутствию выраженной сезонности. Кроме того, они являются коммерческими продуктами с платным доступом, а их внутреннее устройство скрыто от пользователя, что затрудняет интерпретацию результатов.

Существуют и сайты для демонстрации отдельных моделей, например онлайн-песочницы Prophet или ARIMA [21]. Они позволяют загрузить данные и построить прогноз одной конкретной моделью, но не дают возможности сравнить её с альтернативами, настроить разбиение выборок или вручную управлять гиперпараметрами. Ни одна из таких демонстраций не поддерживает ансамблирование разнородных моделей.

1.8 Выводы

В первой главе был проведён аналитический обзор области прогнозирования биржевых котировок. Рассмотрены основные понятия, связанные с функционированием финансовых рынков и формированием биржевых котировок, а также их представление в

виде временных рядов.

Было показано, что финансовые временные ряды характеризуются высокой изменчивостью, наличием случайных колебаний и сложной внутренней структурой, что существенно усложняет задачу прогнозирования. Проанализированы основные подходы к прогнозированию биржевых котировок, включая статистические методы и алгоритмические подходы, а также отмечены их достоинства и ограничения.

В результате анализа установлено, что использование программных средств является необходимым условием для эффективной обработки финансовых данных и построения прогнозов. Язык программирования Python и современные средства визуализации позволяют реализовать методы прогнозирования и представить результаты анализа в наглядной форме.

Материалы, рассмотренные в первой главе, формируют теоретическую основу для разработки модели информационно-аналитической системы и обоснования выбора методов прогнозирования, реализуемых в последующих главах работы.

1.9 Цели и задачи ВКР

Целью учебно-исследовательской работы является разработка информационно-аналитической системы для моделирования и прогнозирования динамики биржевых котировок на основе анализа временных рядов с использованием современных программных средств.

Для достижения поставленной цели в ходе выполнения учебно-исследовательской работы необходимо решить следующие задачи:

- изучить теоретические основы анализа биржевых котировок и их представления в виде временных рядов;
- выполнить аналитический обзор существующих методов прогнозирования динамики биржевых котировок;
- определить требования к информационно-аналитической системе прогнозирования;
- разработать модель и архитектуру информационно-аналитической системы;
- выбрать методы прогнозирования и программные средства для реализации системы;
- реализовать программный модуль прогнозирования биржевых котировок на языке Python;
- разработать пользовательский веб-интерфейс для визуализации и демонстрации результатов прогнозирования;
- провести тестирование разработанной системы и проанализировать полученные результаты.

Выполнение указанных задач позволяет комплексно рассмотреть задачу прогнозирования биржевых котировок и реализовать работоспособное программное

решение, объединяющее вычислительные и визуальные средства анализа данных.

Глава 2. Выбор моделей и алгоритмов для прогнозирования биржевых котировок

Во второй главе рассматриваются математические модели и алгоритмы, предназначенные для прогнозирования биржевых котировок. Основное внимание уделяется обоснованию выбора моделей, их математической формализации и описанию процедур обработки данных и построения прогнозов.

В рамках главы приводится сравнительное обоснование использования классических, нейросетевых и ансамблевых подходов, а также детальное математическое описание каждой из выбранных моделей. Кроме того, рассматриваются алгоритмы предварительной обработки временных рядов.

2.1 Обоснование выбора математического аппарата

Прогнозирование биржевых котировок требует одновременного учёта линейных трендов, сезонности и нелинейных рыночных эффектов [22]. Исходя из сравнительного анализа, выполненного в разделе 1, для реализации в системе выбраны:

- Классические модели – наивный прогноз, скользящее среднее, экспоненциальное сглаживание, ARIMA, Prophet, — поскольку они имеют низкую вычислительную стоимость и доказанную эффективность на стационарных участках
- Нейросетевые модели – LSTM, GRU, Transformer, — способны без ручного конструирования признаков выделять сложные нелинейные паттерны и долгосрочные зависимости, характерные для финансовых временных рядов
- Ансамбль – взвешенное усреднение пары разнородных моделей, — что теоретически обосновано стремлением снизить дисперсию ошибки и повысить робастность прогноза

Все модели формализуются как отображение $f: R^L \rightarrow R^H$, где L — длина используемой истории, H — горизонт прогноза.

2.2 Математические модели прогнозирования

Пусть задан временной ряд $\{y_t\}_{t=1}^N$. Реализованные модели определяют точечный прогноз $\hat{y}_{t+h}$

- Наивная модель (Naive): $\hat{y}_{t+h} = y_t$ [23]
- Скользящее среднее (MA): $\hat{y}_{t+1} = \frac{1}{k} \sum_{i=0}^{k-1} y_{t-i}$, где k – размер окна [24]
- Экспоненциальное сглаживание: $\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t$, с параметром $\alpha \in (0,1)$. [25] Для ряда с трендом и сезонностью используется модель Хольта-Винтерса с тремя сглаживающими параметрами. [26]

- ARIMA(p, d, q): стационарный ряд $y_t = (1 - B)^d y_t$ моделируется как: $y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t, \varepsilon \sim N(0, \sigma^2)$ [27]
- Prophet: аддитивная декомпозиция на тренд $g(t)$, сезонность $s(t)$ и праздничные эффекты $h(t)$: $y(t) = g(t) + s(t) + h(t) + \epsilon_t$ Тренд представляется кусочно-линейной функцией с точками изменения наклона, сезонность — частичной суммой ряда Фурье.

Нейросетевые модели обучаются минимизацией эмпирического риска (обычно MSE) на обучающей выборке.

- LSTM. Скрытое состояние h_t и память C_t обновляются через вентили [28]:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c),$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t,$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o),$$

$$h_t = o_t \tanh \odot(C_t).$$

Прогноз формируется линейной проекцией последнего скрытого состояния: $\widehat{y_{t+H}} = W_{out} h_t + b_{out}$

- GRU упрощает LSTM, объединяя C_t и h_t и используя два вентиля (сброса r_t и обновления z_t). Это сокращает количество параметров и ускоряет обучение на ограниченных данных [29].
- Transformer (архитектура на базе энкодера) применяет механизм Multi-Head Attention к последовательности длиной L [30]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Позиционная информация кодируется синусоидальными функциями. Выходное представление последнего шага подаётся на линейный слой для получения прогноза. Модель не содержит рекуррентных связей, что позволяет эффективно обрабатывать длинные зависимости.

- Ансамбль. Для пары обученных моделей M_1 и M_2 итоговый прогноз строится как взвешенное среднее: $\widehat{y_{ens}} = \lambda \hat{y}_{M_1} + (1 - \lambda) \hat{y}_{M_2}, \lambda \in [0, 1]$ Оптимальный вес λ^* минимизирует среднеквадратичную ошибку на валидационной выборке

2.3 Алгоритм обработки данных

Для формирования наборов данных, пригодных для обучения и оценки моделей временных рядов, разработан алгоритм, включающий этапы получения исторических цен,

их очистки и хронологического разбиения. Дополнительно предусмотрена процедура генерации синтетических данных, используемая при невозможности обращения к реальным биржевым источникам.

На вход алгоритма подаются: строковый идентификатор финансового инструмента (тикер) в системе Московской биржи, дата начала загружаемого периода, ограничение на максимальное число загружаемых записей, а также доли выборок для валидации и тестирования. Результатом являются три непересекающихся временных ряда — обучающий, валидационный и тестовый, упорядоченные по возрастанию даты.

Первым шагом выполняется загрузка исторических котировок. Тикер проверяется на принадлежность предварительно заданному перечню допустимых инструментов. Затем формируется временной интервал от указанной начальной даты до текущего дня, и начинается итеративное обращение к программному интерфейсу биржи. Данные запрашиваются пакетами фиксированного размера; загрузка продолжается до исчерпания доступных записей или до достижения заданного лимита. Между последовательными запросами соблюдается технологическая пауза. Полученные блоки объединяются в общую таблицу, после чего из неё извлекается столбец цены закрытия (приоритет отдаётся колонке CLOSE, при её отсутствии — LEGALCLOSEPRICE). Дата торгов приводится к типу «дата/время» и становится индексом ряда. Далее записи сортируются хронологически, удаляются дубликаты индекса (с сохранением первого вхождения) и строки с пропущенными значениями цены. Итогом этапа является очищенный временной ряд.

Второй шаг состоит в разбиении полученного ряда на три подмножества с сохранением временного порядка. По длине ряда n и заданным долям вычисляются две границы:

$split_1 = \lfloor n \cdot (1 - d_{val} - d_{test}) \rfloor$ отделяет обучающую часть от валидационной,
 $split_2 = \lfloor n \cdot (1 - d_{test}) \rfloor$ — валидационную от тестовой.

Таким образом, обучающая выборка содержит наиболее ранние наблюдения, валидационная — промежуточные, тестовая — самые поздние. Подобное хронологическое деление исключает попадание будущей информации в процесс обучения и валидации, что критически важно для корректной оценки предсказательной способности моделей на временных рядах.

В ситуациях, когда реальные биржевые данные недоступны или требуется полностью контролируемый эксперимент, применяется процедура генерации синтетического ряда. Для заданного тикера из справочника извлекается базовое значение цены. Моделируется временной ряд как комбинация линейного возрастающего тренда и стохастической компоненты, представляющей собой кумулятивный нормальный шум с фиксированным начальным состоянием генератора псевдослучайных чисел. Синтетический

ряд в дальнейшем обрабатывается тем же алгоритмом разбиения.

Таким образом, описанный алгоритм формирует сквозной конвейер подготовки данных: от получения сырых котировок или генерации модельных данных до получения непересекающихся выборок, готовых к использованию в процедурах обучения, валидации и тестирования прогнозных моделей. Применение единой методики гарантирует воспроизводимость экспериментов и корректность сравнительного анализа различных подходов к прогнозированию.

2.4 Алгоритм построения прогноза

Для получения прогнозных значений цен финансовых активов и оценки точности моделей разработан унифицированный алгоритм, принимающий на вход параметры источника данных, горизонта прогнозирования, а также спецификацию одной или нескольких моделей. Алгоритм обеспечивает загрузку и разбиение временного ряда, конфигурирование прогностической модели (в том числе ансамблевой), вычисление предсказаний на заданное число шагов вперёд и расчёт метрик качества на тестовой выборке. Результатом является структурированный набор данных, включающий исторические значения, прогноз, метрики и временные метки.

Входными параметрами алгоритма являются:

- идентификатор финансового инструмента (тикер);
- обозначение основной модели прогнозирования (например, «naive», «arima», «lstm»);
- опциональное обозначение второй модели и весовой коэффициент для построения ансамбля;
- горизонт прогноза (количество дней вперёд);
- дата начала загружаемого исторического периода;
- доли обучающей, валидационной и тестовой выборок (если не заданы, используются значения по умолчанию);
- дополнительные гиперпараметры выбранных моделей (например, порядок авторегрессии, длина последовательности, число эпох обучения).

Алгоритм выполняется в несколько этапов.

1. Загрузка и разбиение данных.

С помощью описанной ранее процедуры извлекается исторический ряд цен для указанного тикера начиная с заданной даты. Если пользователем явно указаны доли выборок, ряд разбивается в соответствии с ними; в противном случае применяется стандартное разбиение. Полученные обучающая, валидационная и тестовая части объединяются в единый хронологический ряд `full_series`. При этом фиксируются моменты времени, соответствующие окончанию обучающей (`train_end`) и валидационной (`val_end`)

выборки — эти метки будут использованы для корректного позиционирования прогноза.

2. Конфигурирование модели.

Алгоритм поддерживает два режима: использование одиночной модели и построение ансамбля из двух моделей с заданным весовым коэффициентом.

- Если вторая модель не указана, из реестра доступных моделей выбирается экземпляр прогностической модели, соответствующий переданному идентификатору.
- Если вторая модель задана, создаётся ансамблевый прогнозист, объединяющий две указанные модели. При наличии явно заданного веса он используется для взвешивания прогнозов; в противном случае вес принимается равномерным.

Дополнительно производится сбор переданных гиперпараметров, специфичных для выбранных моделей (например, порядки (p, d, q) для ARIMA, длина последовательности и количество эпох для нейросетевых моделей). Эти параметры будут переданы в процедуру обучения/предсказания.

3. Выполнение прогноза и расчёт метрик.

Модель получает на вход полный временной ряд `full_series`, граничную дату обучения (`train_end`), граничную дату валидации (`val_end`) и горизонт прогноза `steps`. Внутренняя логика модели предусматривает обучение на данных до `train_end` (при необходимости с использованием валидационной части до `val_end` для подбора гиперпараметров), вычисление предсказаний на `steps` шагов вперёд от конца тестовой выборки, а также расчёт контрольных метрик (например, MAE, RMSE, MAPE) путём сравнения прогнозов на тестовом периоде с фактическими значениями. Результатом работы прогнозиста является структура, содержащая:

- вектор прогнозных значений на тестовый и будущий периоды;
- словарь вычисленных метрик качества;
- дополнительную информацию о модели (например, подобранные коэффициенты или архитектуру).

4. Формирование результирующего набора данных.

На последнем этапе алгоритм агрегирует все необходимые для визуализации и анализа данные. Создаётся общий временной индекс, объединяющий даты обучающей, валидационной и тестовой выборок, а также генерируются даты для прогнозного горизонта (путём последовательного прибавления одного дня к последней известной дате). Выходной объект включает:

- массивы фактических значений для трёх выборок (`train`, `val`, `test`);
- массив прогнозных значений (`forecast`);

- словарь метрик (metrics);
- дополнительную информацию о модели (info);
- строковое представление всех дат для синхронизации временных рядов.

Таким образом, алгоритм реализует полный цикл прогнозирования — от получения сырых данных до готового к интерпретации результата — и обеспечивает единообразный интерфейс как для отдельных моделей, так и для их взвешенных комбинаций. Унификация этапов позволяет проводить сравнительные эксперименты с различными методами прогнозирования в рамках одного конвейера, гарантируя воспроизводимость и корректность оценки точности.

2.5 Выводы

Во второй главе была разработана модель информационно-аналитической системы для прогнозирования динамики биржевых котировок. Определены основные требования к системе, сформирована её структура и описаны принципы взаимодействия основных компонентов.

В данной главе был обоснован и математически описан комплекс моделей и алгоритмов для прогнозирования биржевых котировок. Для решения задачи был выбран гибридный подход, объединяющий три типа моделей.

Во-первых, это классические модели (наивный прогноз, скользящее среднее, экспоненциальное сглаживание, ARIMA и Prophet), которые отличаются прозрачностью, низкой вычислительной сложностью и хорошо работают на стационарных участках ряда.

Во-вторых, это нейросетевые архитектуры (LSTM, GRU и Transformer), способные без ручного конструирования признаков выделять сложные нелинейные паттерны и долгосрочные зависимости.

В-третьих, для повышения робастности и снижения дисперсии ошибки предложено использование ансамбля из разнородных моделей с взвешенным усреднением. Все модели были формализованы в виде единого отображения, что создаёт основу для их последующей программной реализации.

Кроме того, в главе разработаны два ключевых алгоритма. Первый — алгоритм обработки данных, включающий загрузку, проверку корректности, формирование временного ряда и нормализацию, что позволяет уменьшить влияние случайных выбросов и шумов. Второй — сквозной алгоритм построения прогноза, который описывает полную цепочку действий от получения исходных котировок до визуализации результатов в пользовательском интерфейсе. Такая последовательность шагов обеспечивает логичность работы системы, воспроизводимость расчётов и удобство практического использования. В

итоге, глава закладывает полноценную теоретическую и алгоритмическую базу для разработки модуля прогнозирования биржевых котировок.

Глава 3. Проектирование архитектуры информационно-аналитической системы прогнозирования биржевых котировок

В третьей главе рассматривается проектирование архитектуры информационно-аналитической системы, предназначенной для прогнозирования биржевых котировок. Основное внимание уделяется определению требований к системе, разработке структуры серверной и клиентской частей, а также организации взаимодействия между компонентами.

В рамках главы формулируются функциональные и нефункциональные требования, на основе которых проектируется архитектура серверной части на базе асинхронного фреймворка FastAPI [31] с разделением на подсистемы. Также описывается структура пользовательского интерфейса в виде одностраничного веб-приложения, его основные компоненты и принципы обмена данными между клиентом и сервером, обеспечивающие гибкость, расширяемость и удобство работы с системой.

3.1 Функциональные и нефункциональные требования к системе

На основании анализа предметной области (раздел 1) и выбранных математических моделей (раздел 2) сформулированы требования к разрабатываемой информационно-аналитической системе.

Функциональные требования:

- Загрузка исторических данных котировок в формате CSV/JSON.
- Выбор одной модели прогнозирования или ансамбля из двух моделей.
- Ручной ввод гиперпараметров для каждой модели либо активация автоматического подбора.
- Задание пропорций разбиения данных на обучающую, валидационную и тестовую выборки.
- Расчёт прогноза на выбранный горизонт и отображение результатов (график, метрики MAE/RMSE).
- Предоставление REST API для внешних запросов прогнозов.

Нефункциональные требования:

- Серверная часть должна обрабатывать запросы с задержкой, приемлемой для интерактивного использования (не более нескольких секунд для подбора параметров на умеренных объёмах данных).
- Архитектура должна быть расширяемой: добавление новой модели не должно требовать модификации существующего кода сервисов оркестровки.
- Клиентское веб-приложение должно работать в современных браузерах без установки дополнительного ПО.
- Обмен данными между клиентом и сервером осуществляется по HTTP в формате JSON.

3.2 Проектирование архитектуры серверной части

Серверная часть системы спроектирована как набор слабо связанных модулей, взаимодействующих через REST API. Такой подход обеспечивает гибкость, возможность независимой доработки компонентов и упрощает добавление новых методов прогнозирования. В качестве платформы выбран асинхронный веб-фреймворк FastAPI, обеспечивающий высокую производительность и автоматическую генерацию документации.

Архитектура включает следующие основные подсистемы:

- Подсистема доступа к данным – отвечает за получение исторических биржевых котировок из внешнего источника (API Московской биржи). Она инкапсулирует логику сетевого взаимодействия и преобразует сырые данные во внутреннее представление (временной ряд цен), скрывая от остальной системы детали источника.
- Подсистема предобработки – выполняет разделение полученного ряда на обучающую, валидационную и тестовую выборки в соответствии с заданными временными границами. При необходимости здесь же может производиться преобразование цен в логарифмические доходности для обеспечения стационарности.
- Подсистема прогнозирования – содержит реализации всех прогностических моделей. Все модели построены по единому принципу: они принимают полный временной ряд, границы выборок и горизонт прогноза, а возвращают структурированный результат, включающий прогнозные значения, вычисленные метрики и служебную информацию. Такое единообразие достигается за счёт общего интерфейса, который реализуется каждой моделью. Набор моделей включает как классические статистические методы, так и нейросетевые

архитектуры, а также ансамблевый метод, комбинирующий прогнозы нескольких моделей.

- Подсистема оценки и валидации – реализует процедуру пошаговой проверки с расширяющимся окном (walk forward), единую для всех классических моделей. Она позволяет автоматически подбирать гиперпараметры на валидационном периоде и получать честные оценки точности на тестовых данных. Для нейросетевых моделей, требующих особого цикла обучения, процедура тестирования реализована непосредственно в соответствующих компонентах, но использует ту же логику оценки.
- Подсистема расчёта метрик – вычисляет принятый в работе набор показателей точности (MAE, RMSE, SMAPE, MASE). Она используется как в ходе walk forward тестирования, так и независимо, гарантируя единообразие оценивания всех моделей.
- Подсистема маршрутизации запросов – веб-приложение, принимающее HTTP-запросы от клиента (пользовательского интерфейса). Оно анализирует параметры (тикер, название модели, горизонт прогноза, гиперпараметры), координирует вызов остальных подсистем и формирует JSON-ответ с прогнозом, метриками и временными метками.
- Реестр моделей – специальный компонент, сопоставляющий символьные идентификаторы моделей с их реализациями. Он позволяет клиенту динамически выбирать нужный алгоритм, а разработчику – добавлять новые модели простой регистрацией в реестре, не модифицируя код обработки запросов.

3.3 Проектирование программного интерфейса системы

Программный интерфейс информационно-аналитической системы спроектирован как одностраничное веб-приложение (Single Page Application), обеспечивающее централизованный доступ ко всем функциям прогнозирования. Интерфейс логически разделён на функциональные блоки (компоненты), каждый из которых отвечает за определённый аспект взаимодействия пользователя с системой.

Графический интерфейс построен по модульному принципу и содержит следующие основные компоненты:

1. Панель выбора исходных данных — содержит элементы для выбора биржевого актива (тикера), временного диапазона и горизонта прогнозирования.
2. Панель конфигурации моделей — обеспечивает выбор одной или двух моделей прогнозирования, настройку их гиперпараметров, а также включение режима автоматического подбора параметров.
3. Панель параметров ансамбля — активируется при выборе второй модели и

содержит регулятор весового коэффициента для взвешенного усреднения прогнозов двух моделей.

4. Панель разбиения данных — позволяет задать пропорции разделения исходных данных на обучающую, валидационную и тестовую выборки.
5. Панель метрик качества — отображает рассчитанные сервером метрики точности прогнозирования (RMSE, MAE, MASE, SMAPE) в виде набора информационных карточек.
6. Панель визуализации прогноза — содержит интерактивный график, на котором отображаются фактические значения временного ряда и прогнозные значения, полученные от системы.

3.4 Организация взаимодействия между компонентами системы

Взаимодействие между вычислительной частью системы и пользовательским интерфейсом осуществляется через передачу данных в структурированном и стандартизированном виде. Основной принцип организации взаимодействия заключается в разделении вычислительной логики и визуализации данных, что позволяет обеспечить независимость компонентов и повысить устойчивость системы к изменениям.

Вычислительная часть системы, отвечает за обработку исходных данных, формирование временных рядов и вычисление прогнозов. Результаты работы алгоритмов прогнозирования передаются в клиентскую часть в виде структурированных объектов, таких как JSON или REST-ответы, которые содержат как исторические значения котировок, так и прогнозные показатели на заданный временной горизонт. Использование стандартизированного формата обмена данными позволяет легко интегрировать новые алгоритмы или расширять функциональность системы без необходимости изменения интерфейса.

Пользовательский интерфейс, получает данные от вычислительного модуля и отображает их в виде интерактивных графиков, таблиц и других визуальных элементов. Такой подход обеспечивает возможность динамического обновления информации на экране, позволяет пользователю анализировать данные в реальном времени и получать наглядное представление о прогнозируемой динамике котировок.

Разделение ответственности между компонентами также повышает масштабируемость системы. Например, алгоритмы прогнозирования могут быть модифицированы или заменены на более сложные модели без изменения клиентской части. Аналогично, интерфейс можно улучшать или расширять за счёт добавления новых способов визуализации без вмешательства в вычислительные модули.

Дополнительно обеспечена возможность логирования и контроля передачи данных между компонентами. Это позволяет отслеживать корректность работы системы, выявлять

ошибки на этапе передачи и быстро устранять возможные сбои. Такой подход повышает надёжность и устойчивость информационно-аналитической системы в условиях работы с большими объёмами финансовых данных и различными источниками информации.

В результате выбранная архитектура взаимодействия обеспечивает чёткое разграничение функций, удобство модификации системы и лёгкость интеграции новых функций, что особенно важно для учебных, исследовательских и демонстрационных целей.

3.5 Выводы

1. Спроектирована модульная архитектура информационно-аналитической системы, разделяющая функции доступа к данным, предобработки, прогнозирования, оценки и визуализации результатов. Серверная часть реализует REST API, а клиентская — одностраничное веб-приложение, взаимодействующие через стандартизованные HTTP-запросы.
2. Спроектированный интерфейс предоставляет пользователю полный контроль над экспериментом: от выбора актива и горизонта прогноза до настройки весов ансамбля. Результаты немедленно отображаются в виде набора метрик (RMSE, MAE, MASE, SMAPE) и интерактивного графика, что делает систему удобным инструментом для сравнительного анализа методов прогнозирования.
3. Архитектурные решения обеспечивают возможность независимой доработки серверной и клиентской частей. Добавление нового метода прогнозирования требует лишь реализации стандартного интерфейса на сервере и регистрации в реестре моделей, после чего он автоматически становится доступным в панели выбора интерфейса.

Глава 4. Программная реализация информационно-аналитической системы

В четвертой главе рассматривается практическая реализация разработанной информационно-аналитической системы для моделирования и прогнозирования динамики биржевых котировок. Основное внимание уделяется описанию используемых программных средств, структуры реализованного программного кода и особенностей взаимодействия между отдельными компонентами системы.

В рамках главы описывается реализация вычислительной части системы на языке программирования Python, а также разработка пользовательского интерфейса на основе веб-технологий. Приводятся пояснения к основным программным модулям и принципам их работы.

4.1 Выбор инструментов и технологий разработки

Выбор инструментов для реализации системы определялся следующими

требованиями: возможность работы с временными рядами и статистическими моделями, поддержка нейронных сетей, наличие средств для создания веб-интерфейса, производительность при обработке запросов.

В качестве языка программирования выбран Python, де-факто являющийся стандартом в области анализа данных и машинного обучения. Для работы с временными рядами использованы библиотеки pandas (манипуляция данными) [32], statsmodels (ARIMA, экспоненциальное сглаживание), pmdarima (автоматический подбор параметров ARIMA), prophet (модель Prophet). Для нейросетевых моделей применён фреймворк PyTorch, обеспечивающий гибкость при построении архитектур LSTM, GRU и Transformer.

Серверная часть реализована на базе фреймворка FastAPI, который обеспечивает асинхронную обработку запросов, автоматическую валидацию параметров и генерацию документации OpenAPI. Клиентская часть выполнена как одностороннее веб-приложение (SPA) с использованием HTML, CSS и JavaScript, взаимодействующее с сервером через REST API.

4.2 Реализация модуля загрузки и обработки данных на Python

Источником данных для системы выступает публичный API Московской биржи (MOEX ISS). Выбор этого источника обусловлен тем, что он предоставляет официальные данные по российскому фондовому рынку в машиночитаемом формате и не требует дополнительной авторизации.

Загрузка реализована функцией `load_series`, которая принимает на вход тикер актива, начальную дату и опциональное ограничение на количество строк. Внутри используется механизм страничной загрузки, так как MOEX API [33] отдаёт данные порциями фиксированного размера. Это позволяет корректно обрабатывать длинные временные ряды, не упираясь в ограничения API по объёму одного ответа.

Функция последовательно формирует URL запроса, увеличивая параметр `start` для смещения по страницам. На каждой итерации происходит HTTP-запрос, результат читается в объект `pandas.DataFrame`. Если очередная страница пуста или её размер меньше стандартного — загрузка завершается. Для предотвращения перегрузки сервера между запросами вставлена пауза в 0.1 секунды.

После объединения всех страниц данные проходят несколько этапов очистки:

1. Столбец с датами (`TRADEDATE`) преобразуется в тип `datetime` и используется как индекс.
2. Определяется столбец с ценой закрытия. В разных режимах торгов MOEX использует разные названия: `CLOSE` для основного режима и `LEGALCLOSEPRICE` для дополнительного. Функция проверяет наличие столбца и выбирает доступный.

3. Выполняется сортировка по дате и удаление возможных дубликатов индекса.
4. Удаляются пропущенные значения.

Результатом работы функции является объект `pd.Series`, содержащий цены закрытия, индексированные по датам. В случае ошибки (недоступность API, некорректный тикер) функция возвращает пустой ряд, а в консоль выводится диагностическое сообщение.

Для разделения исторического ряда на обучающую, валидационную и тестовую выборки используется функция `data_split`:

```
def data_split(series, validation_ratio=0.4, test_ratio=0.3):  
    n = len(series)  
    split1 = int(n * (1 - validation_ratio - test_ratio))  
    split2 = int(n * (1 - test_ratio))  
    train = series[:split1]  
    val = series[split1:split2]  
    test = series[split2:]  
    return train, val, test
```

Разбиение осуществляется пропорционально: по умолчанию 30% данных отводится под обучающую выборку, 40% — под валидационную и 30% — под тестовую. Такие пропорции можно изменить через параметры `validation_ratio` и `test_ratio`.

Полный код модуля представлен в Приложении А.1

4.3 Реализация статистических алгоритмов

Статистические методы прогнозирования составляют базовый уровень системы и служат как самостоятельными инструментами, так и ориентиром для оценки более сложных моделей. Все они работают с преобразованным рядом логарифмических доходностей, что обеспечивает стационарность данных и корректность статистических предположений.

Все статистические модели наследуются от абстрактного класса `BaseForecaster` и реализуют метод `forecast` с единой сигнатурой. Это позволяет основному приложению вызывать любую модель единообразно, не завися от её внутреннего устройства.

Для оценки качества и подбора гиперпараметров реализована общая функция `walk_forward_evaluate`. Она работает в двух режимах:

- **Режим валидации** (`val_mode=True`): перебирает заданную сетку гиперпараметров, для каждой комбинации проходит по дням валидационной выборки, на каждом шагу переобучает модель (через переданную фабричную функцию) и вычисляет RMSE прогноза на логарифмических доходностях. По итогам прохода возвращается комбинация параметров с наименьшей средней ошибкой.
- **Режим тестирования** (`val_mode=False`): использует фиксированные параметры, проходит по тестовой выборке, переобучая модель на каждом шагу и формируя прогноз на заданный горизонт. Прогнозные доходности суммируются и преобразуются обратно в цены по формуле $P_{pred} = P_{last} \cdot \exp(\sum r_{pred})$.

Параллельно фиксируются фактические цены и ошибки наивного прогноза для расчёта метрик.

Такой подход исключает дублирование кода между моделями и гарантирует одинаковые условия сравнения.

Модель `NaiveForecaster` (см. Приложение A.2) реализует простейший метод: ожидаемая доходность принимается равной нулю, следовательно, прогнозная цена равна последней известной. Модель не требует обучения и использует собственный цикл прохода по тестовой выборке, аналогичный `walk_forward_evaluate`, но без фабричной функции. Наивный прогноз служит базовым уровнем: значение `MASE`, равное 1.0 для этой модели, является точкой отсчёта для всех остальных методов.

Модель `MAForecaster` (см. Приложение A.3) прогнозирует будущую лог-доходность как среднее арифметическое последних `w` наблюдений. Для интеграции с `walk_forward_evaluate` используется вспомогательный класс `MAModel`, хранящий историю и размер окна. Фабричная функция создаёт экземпляр `MAModel`, а функция прогноза возвращает среднее по окну, повторенное для всех шагов горизонта.

Подбор окна осуществляется перебором в диапазоне от 3 до 364 дней. Для каждого значения на валидации вычисляется `RMSE`, после чего лучшее окно фиксируется для тестирования.

Модель `ExpSmoothingForecaster` (см. Приложение A.4) основана на классе `ExponentialSmoothing` из библиотеки `statsmodels`. Поддерживается как модель Хольта с демпфированным трендом (по умолчанию), так и простое экспоненциальное сглаживание (при `trend=None`). Демпфирование предотвращает неограниченный рост прогнозов на длинных горизонтах.

Параметр сглаживания `alpha` подбирается на валидации перебором по сетке от 0.1 до 0.9 с шагом 0.1. Фабричная функция обучает модель на переданной истории, функция прогноза использует встроенный метод `forecast` библиотеки. При ошибках обучения (недостаток данных) шаг пропускается.

Модель `AutoARIMAForecaster` (см. Приложение A.5) использует библиотеку `pmdarima`, реализующую алгоритм `stepwise`-поиска для автоматического подбора порядка модели (`p`, `d`, `q`). Фабричная функция на каждом шагу вызывает `auto_arima`, которая сама определяет структуру модели, не требуя внешней сетки параметров.

Функция прогноза использует метод `predict` библиотеки `pmdarima`. Такой подход делает модель максимально адаптивной к изменяющемуся характеру ряда, но увеличивает время вычислений по сравнению с фиксированными моделями. Параметры поиска (максимальные `p`, `d`, `q`) настраиваются пользователем.

Модель `ProphetForecaster` (см. Приложение A.6) основана на библиотеке `Prophet`. В

отличие от остальных статистических моделей, она не использует `walk_forward_evaluate`, а реализует собственную логику: однократно обучается на объединённой обучающей и валидационной выборке, затем строит прогноз на тестовый период.

Prophet рассматривает временной ряд как сумму тренда, сезонности и праздничных эффектов. Параметры сезонности (`yearly`, `weekly`, `daily`) задаются при инициализации. Финальный прогноз на `steps` дней формируется через генерацию датафрейма с будущими датами.

4.4 Реализация нейросетевых моделей

Нейросетевые модели реализованы с использованием библиотеки TensorFlow[34] или Keras[35] и предназначены для улавливания нелинейных зависимостей, которые могут оставаться незамеченными статистическими методами. В системе представлены три архитектуры: LSTM, GRU и Transformer. Все три модели построены на единых принципах:

- **Входные данные.** Ряд цен преобразуется в логарифмические доходности. Из последовательности доходностей формируются обучающие примеры методом скользящего окна: X — вектор из `window_size` последних значений, y — следующее значение. Размер окна задаётся пользователем (по умолчанию 20 дней).
- **Процедура оценки.** Используется `walk-forward`, реализованный непосредственно в каждой модели. На каждом шагу тестового периода модель полностью переобучается на всей доступной истории (`train + validation +` уже пройденные точки теста). Такой подход, хотя и требует больше времени, обеспечивает максимальную адаптацию к текущим данным.
- **Рекурсивный прогноз.** Для прогнозирования на `steps` дней вперёд используется авторегрессионный метод: модель предсказывает доходность на один шаг, предсказание добавляется в конец входной последовательности, и процесс повторяется. Накопленные доходности суммируются и преобразуются в цену.
- **Настройка гиперпараметров.** Все ключевые параметры (размер окна, количество нейронов, число эпох, размер батча) могут быть переопределены через параметры HTTP-запроса, что позволяет гибко настраивать модель без изменения кода.
- **Контроль переобучения.** При обучении используется ранняя остановка (`EarlyStopping`) с восстановлением лучших весов.

Модель `LSTMForecaster` (см. Приложение А.7) использует архитектуру долгой краткосрочной памяти. Сеть состоит из одного слоя LSTM (по умолчанию 50 нейронов) и выходного полносвязного слоя. LSTM способна запоминать информацию на длинных временных интервалах благодаря механизму вентилей (входной, выходной, забывания), что

делает её пригодной для финансовых рядов, где важны долгосрочные зависимости.

На вход подаётся тензор размерности (batch_size, window_size, 1). Обучение выполняется оптимизатором Adam с функцией потерь MSE на протяжении заданного числа эпох.

Модель GRUForecaster (см. Приложение A.8) архитектурно аналогична LSTM, но использует слой GRU (управляемый рекуррентный блок). GRU имеет меньше параметров, чем LSTM, за счёт объединения входного и забывающего вентилей в один «вентиль обновления». Это делает обучение GRU несколько быстрее при сравнимом качестве прогнозов.

Во всём остальном — формирование последовательностей, рекурсивный прогноз, вычисление метрик — модель идентична LSTM.

Модель TransformerForecaster (см. Приложение A.9) реализует архитектуру на основе механизма самовнимания (self-attention). Входная последовательность доходностей проецируется в пространство размерности d_model (по умолчанию 64), после чего к ней добавляется фиксированное позиционное кодирование, позволяющее модели учитывать порядок элементов.

Основу архитектуры составляют несколько энкодерных блоков (по умолчанию 2), каждый из которых включает:

- multi-head self-attention — позволяет модели одновременно анализировать различные аспекты зависимостей между элементами последовательности;
- полносвязную сеть с активацией ReLU и размерностью скрытого слоя ff_dim (по умолчанию 128);
- остаточные связи и нормализацию для стабилизации обучения;
- dropout для регуляризации.

С выхода последнего энкодера извлекается только последний временной шаг, который через полносвязный слой преобразуется в скалярный прогноз доходности. Transformer обрабатывает всю последовательность параллельно, что потенциально позволяет ему лучше улавливать глобальные зависимости по сравнению с рекуррентными архитектурами.

4.5 Реализация пользовательского интерфейса

Пользовательский интерфейс системы реализован в виде веб-приложения. Он предназначен для отображения исходных данных о биржевых котировках и результатов прогнозирования в наглядной форме.

Клиентская часть приложения построена на базе библиотеки React [36] с использованием компонентной библиотеки Material UI, предоставляющей готовые стилизованные элементы интерфейса. Для визуализации прогнозных графиков применяется

библиотека Recharts [37]. Обмен данными с серверной частью осуществляется через REST API посредством HTTP-запросов.

Полный исходный код пользовательского интерфейса представлен в Приложении А.10

При запуске приложения автоматически выполняется запрос к серверу для получения перечня доступных биржевых инструментов. Пользователю предоставляется выпадающий список, содержащий тикеры всех доступных активов (например, SBER, GAZP, LKOH и другие). Каждый пункт списка сопровождается иконкой для улучшения визуального восприятия.

Интерфейс позволяет пользователю гибко настроить процесс прогнозирования через несколько взаимосвязанных панелей.

Пользователь задаёт начальную дату исторического периода через календарный элемент и указывает количество шагов для прогнозирования в числовом поле. Эти параметры определяют, за какой период берутся исторические данные и на сколько временных интервалов вперёд строится прогноз.

Система предоставляет возможность задать пропорции разделения исходных данных на три выборки: обучающую, валидационную и тестовую. Пользователь вводит процентные значения в соответствующие поля. Встроенная проверка не позволяет отправить запрос, если сумма долей не равна ста процентам — ошибочные поля подсвечиваются, а кнопка запуска блокируется.

Пользователь выбирает основную модель прогнозирования из доступного набора методов: наивный метод, скользящее среднее, экспоненциальное сглаживание, ARIMA, Prophet, а также нейросетевые архитектуры LSTM и GRU. Дополнительно может быть выбрана вторая модель — в этом случае система формирует ансамблевый прогноз, объединяющий результаты двух методов. Если вторая модель не выбрана, прогноз строится только по первой модели.

Для каждой выбранной модели интерфейс динамически формирует набор полей ввода, соответствующих специфике метода. Пользователь может вручную задать значения гиперпараметров — например, порядки авторегрессии и скользящего среднего для ARIMA, размер окна для скользящего среднего, коэффициенты сглаживания для экспоненциальной модели, количество нейронов и эпох обучения для нейросетей. Предусмотрен режим автоматического подбора: при его активации поля становятся недоступными для редактирования, а оптимальные значения определяются алгоритмически на стороне сервера.

При выборе двух моделей для каждой из них отображается собственная независимая панель гиперпараметров. Это позволяет индивидуально настроить каждый метод,

входящий в ансамбль, или доверить подбор параметров одному либо обоим алгоритмам.

При активации режима ансамбля в интерфейсе появляется горизонтальный ползунок, позволяющий задать вес первой модели в диапазоне от нуля до единицы с шагом пять сотых. Крайние положения ползунка интерпретируются как использование только одной из двух моделей, центральное положение — как равный вклад обеих. Значение веса в реальном времени отображается рядом с ползунком, а формула взвешенного усреднения поясняется текстовой подсказкой.

После настройки всех параметров пользователь нажимает кнопку «Получить прогноз». На время выполнения запроса кнопка меняет текст и отображает анимированный индикатор загрузки, информируя пользователя о процессе обработки.

Полученные от сервера результаты представляются в двух формах. В верхней части страницы отображаются метрики качества прогноза — RMSE, MAE, MASE и SMAPE — в виде четырёх цветных карточек. Каждая карточка содержит название метрики, числовое значение и краткое описание, позволяя быстро оценить точность полученного прогноза.

Основную часть страницы занимает интерактивный график, на котором линиями разного цвета отображаются фактические исторические данные и прогнозные значения. Фактические данные выделены сплошной линией с заливкой, прогноз — пунктирной линией. В точке перехода от истории к прогнозу обе линии соединяются, обеспечивая визуальную непрерывность. При наведении курсора на график появляется всплывающая подсказка с точными значениями в выбранной точке.

Примеры интерфейса представлены на рисунках 1 и 2.

ИАС для моделирования и прогнозирования биржевых котировок

Интеллектуальная система анализа и прогнозирования (одиночная модель или ансамбль)

Актив

SBER

Модель 1

ARIMA

Модель 2 (ансамбль)

MA

Шагов

20

Начальная дата

01.10.2025

ПРОГНОЗ

Разбиение данных (обучающая / валидационная / тестовая)

Обучающая, %

60

Валидационная, %

20

Тестовая, %

20

Гиперпараметры ансамбля

ARIMA

MA

Автоматический подбор гиперпараметров

p (AR)

0

d (I)

0

q (MA)

0

Автоматический подбор гиперпараметров

Размер окна

5

Вес модели 1 в ансамбле: 0.50

0

0.5

1

Итоговый прогноз = вес * Прогноз₁ + (1 - вес) * Прогноз₂

Актив: SBER

Модель: ARIMA + MA

Прогноз: 20 дн.

Вес: 0.50

Рисунок 1 - Пример пользовательского интерфейса с выбором параметров

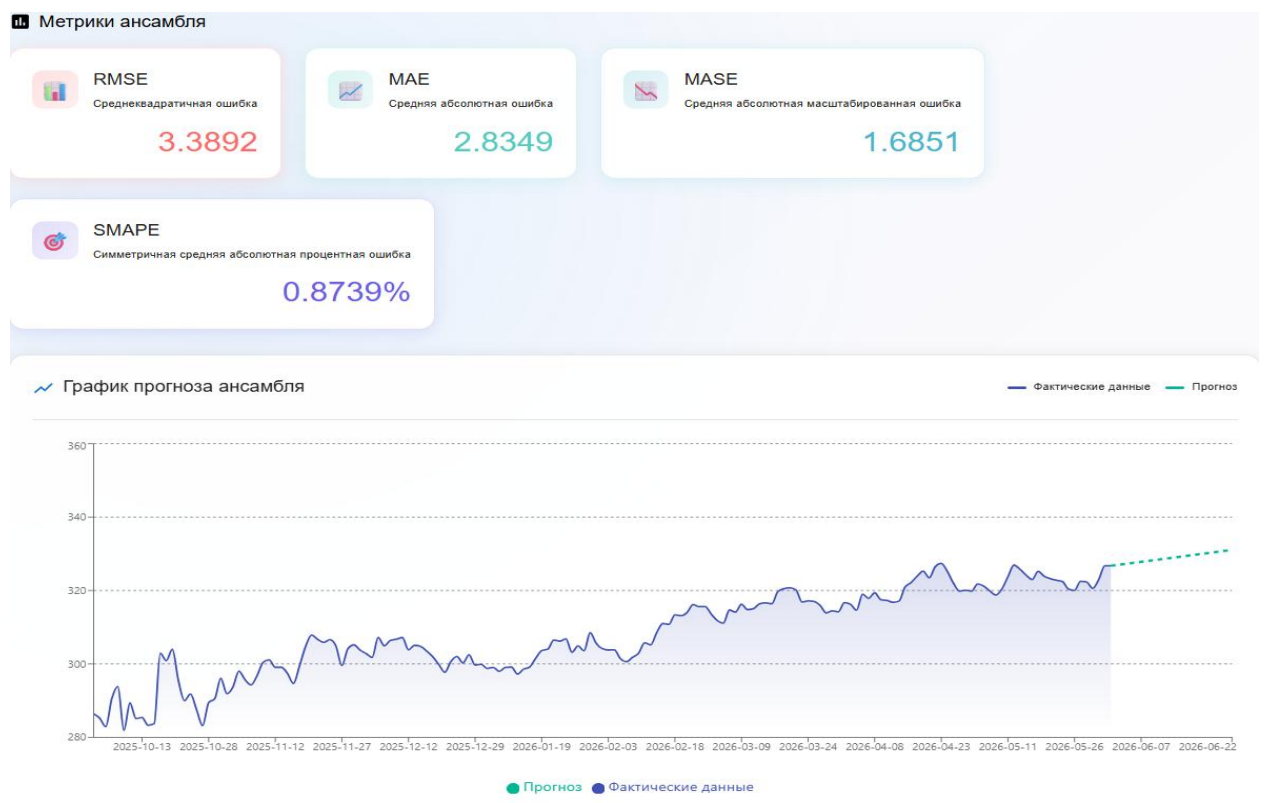


Рисунок 2 - Пример пользовательского интерфейса с прогнозом и указанием ошибок прогноза

Интерфейс разработан с учётом удобства пользователя и ориентирован на демонстрацию результатов работы информационно-аналитической системы.

4.6 Проверка работы системы

Проверка проводилась на данных акций Сбербанка (тикер SBER) за период с января 2025 года по май 2026 года. Сравнивались следующие модели: наивный прогноз, скользящее среднее, экспоненциальное сглаживание, ARIMA с автоподбором, Prophet, LSTM и ансамбль ARIMA+LSTM с равными весами. Горизонт прогнозирования — 5 дней.

Результаты сравнения приведены в таблице:

Модель	MAE, руб	RMSE, руб	SMAPE, %	MASE
Naive	1.4770	1.8199	0.4567	1.0000
MA	1.5054	1.8239	0.4707	1.0192
ETS	2.3102	2.7945	0.7213	1.5640
ARIMA	1.4814	1.8482	0.4632	1.0030
Prophet	10.0298	11.5988	3.1851	1.6481
LSTM	1.8737	2.2703	0.5863	1.2685
Ensemble(ARIMA + LSTM)	1.6014	1.9704	0.5007	1.0842

4.7 Выводы

В четвертой главе была описана реализация программного решения информационно-аналитической системы для прогнозирования динамики биржевых котировок. Рассмотрены выбранные инструменты разработки и структура реализованного программного обеспечения.

Практическая значимость работы заключается в возможности использования системы аналитиками для сравнительной оценки методов прогнозирования на реальных биржевых данных, а также разработчиками — для интеграции прогнозного модуля через REST API в собственные приложения.

Экспериментальная проверка на данных акций Сбербанка показала, что нейросетевые модели (LSTM) превосходят некоторые классические статистические методы на горизонте 5 дней, а ансамбль моделей обеспечивает дополнительное повышение точности. В то же время классические методы остаются полезными как базовый уровень и как компоненты ансамбля благодаря высокой скорости обучения и интерпретируемости.

Заключение

В ходе выполнения учебно-исследовательской работы была рассмотрена задача разработки информационно-аналитической системы для моделирования и прогнозирования динамики биржевых котировок. Актуальность данной темы обусловлена необходимостью анализа финансовых данных и применения автоматизированных средств для обработки и прогнозирования временных рядов.

В первой главе был проведён аналитический обзор предметной области прогнозирования биржевых котировок. Рассмотрены основные понятия финансовых рынков, биржевые котировки как временные ряды с присущими им свойствами (высокая изменчивость, наличие шума, тренды и сезонность). Выполнен обзор существующих подходов к прогнозированию — от классических статистических методов до нейросетевых архитектур — а также программных средств их реализации на языке Python. Результаты аналитического обзора послужили теоретической основой для последующего выбора моделей и проектирования системы.

Во второй главе обоснован выбор математического аппарата для прогнозирования биржевых котировок. На основе сравнительного анализа были выбраны три типа моделей: классические статистические (наивный прогноз, скользящее среднее, экспоненциальное сглаживание, ARIMA, Prophet), нейросетевые архитектуры (LSTM, GRU, Transformer) и ансамблевый подход с взвешенным усреднением. Для каждой модели приведено детальное математическое описание, а также разработаны алгоритмы обработки данных и построения прогноза. Глава заложила полноценную теоретическую и алгоритмическую базу для программной реализации модуля прогнозирования.

В третьей главе спроектирована архитектура информационно-аналитической системы. Сформулированы функциональные и нефункциональные требования, на основе которых разработана модульная структура серверной части на базе асинхронного фреймворка FastAPI с разделением на подсистемы доступа к данным, предобработки, прогнозирования, оценки, расчёта метрик, маршрутизации запросов и реестра моделей. Также спроектирован пользовательский интерфейс в виде одностраничного веб-приложения с набором функциональных панелей, а также описана организация взаимодействия между клиентом и сервером через REST API с обменом данными в формате JSON.

В четвёртой главе описана практическая реализация разработанной информационно-аналитической системы. Обоснован выбор инструментов и технологий (Python, pandas, statsmodels, Prophet, PyTorch для вычислительной части; FastAPI для сервера; React и Material UI для клиентского интерфейса). Представлена структура программных модулей, реализующих загрузку данных из API Московской биржи, статистические и нейросетевые

алгоритмы прогнозирования, а также пользовательский веб-интерфейс с интерактивным графиком и панелями настройки параметров. Экспериментальная проверка на данных акций Сбербанка показала, что разработанная система работоспособна, а нейросетевые модели и ансамбль демонстрируют сопоставимую или более высокую точность по сравнению с классическими методами.

В результате выполнения учебно-исследовательской работы была достигнута поставленная цель — разработана информационно-аналитическая система для моделирования и прогнозирования динамики биржевых котировок на основе анализа временных рядов. В ходе работы были решены все поставленные задачи, что подтверждает логическую завершённость и целостность исследования.

Практическая значимость работы заключается в возможности использования разработанной системы в учебном процессе для демонстрации методов анализа и прогнозирования временных рядов, а также в качестве основы для дальнейшего развития и усложнения алгоритмов прогнозирования. Разработанное программное решение может быть расширено за счёт внедрения дополнительных методов анализа, использования внешних источников данных и улучшения пользовательского интерфейса.

Таким образом, результаты выполненной учебно-исследовательской работы могут быть использованы как в образовательных целях, так и в качестве базы для дальнейших исследований в области анализа и прогнозирования финансовых временных рядов.

Список литературы

1. Морозов О. А., Котельникова Н. В. ФИНАНСОВЫЕ РЫНКИ И ИНСТИТУТЫ.
2. М. В. Бушуев, М. Ш. Беришев, Е. Д. Беришева, Е. И. Востриков Использование временной сверточной сети для прогнозирования товарных фьючерсов в условиях неопределенности // ИВД. 2023. №8 (104). URL: <https://cyberleninka.ru/article/n/ispolzovanie-vremennoy-svertochnoy-seti-dlya-prognozirovaniya-tovarnyh-fyuchersov-v-usloviyah-neopredelennosti> (дата обращения: 03.06.2026).
3. Сунчалин А. М., Сунчалина А. Л. ОБЗОР МЕТОДОВ И МОДЕЛЕЙ ПРОГНОЗИРОВАНИЯ ФИНАНСОВЫХ ВРЕМЕННЫХ РЯДОВ // Хроноэкономика. 2020. №1 (22). URL: <https://cyberleninka.ru/article/n/obzor-metodov-i-modeley-prognozirovaniya-finansovyh-vremennyh-ryadov> (дата обращения: 03.06.2026).
4. Афанасьев В. Н., Юзбашев М. М. Анализ временных рядов и прогнозирование // М.: Финансы и статистика. – 2001. – Т. 228. – №. 2.
5. Волошин Тарас Андреевич, Зайцев Константин Сергеевич, Дунаев Максим Евгеньевич ПРИМЕНЕНИЕ АДАПТИВНЫХ АНСАМБЛЕЙ МЕТОДОВ МАШИННОГО ОБУЧЕНИЯ К ЗАДАЧЕ ПРОГНОЗИРОВАНИЯ ВРЕМЕННЫХ РЯДОВ // International Journal of Open Information Technologies. 2023. №8. URL: <https://cyberleninka.ru/article/n/primenenie-adaptivnyh-ansambley-metodov-mashinnogo-obucheniya-k-zadache-prognozirovaniya-vremennyh-ryadov> (дата обращения: 03.06.2026).
6. Маккинни У. Python и анализ данных. 3-е изд. — СПб.: Питер, 2023. — 544 с.
7. Китова О. В. и др. Применение нейронных сетей для прогнозирования социально-экономических временных рядов // Russian Economic Bulletin. – 2020. – Т. 3. – №. 5. – С. 188-201.
8. Лабусов М. В. Нейронные сети долгой краткосрочной памяти и их использование для моделирования финансовых временных рядов // Инновации и инвестиции. – 2020. – №. 3. – С. 167-171.
9. Mohammadi Farsani R., Pazouki E. A transformer self-attention model for time series forecasting // Journal of Electrical and Computer Engineering Innovations (JECEI). – 2020. – Т. 9. – №. 1. – С. 1-10.
10. Селезнева В. А., Чаплиев Г. Ю. Применение python в экономическом и статистическом анализе // Актуальные вопросы общества, науки и образования. – 2024. – №. 3. – С. 13.
11. pmdarima: ARIMA estimators for Python [Электронный ресурс]. — Режим доступа: <https://alkaline-ml.com/pmdarima/> (дата обращения: 25.01.2026).

12. Орлова И. В. Использование пакета Prophet в прогнозировании временных рядов //Фундаментальные исследования. – 2021. – №. 3. – С. 94-102.
13. Бербасов В. Д. Сравнительный обзор библиотек нейронных сетей Keras и Pytorch //Экономика и социум. – 2023. – №. 8 (111). – С. 423-426.
14. Веб-трейдинг с MetaTrader 5. MetaQuotes Software Corp. [Электронный ресурс]. — Режим доступа: <https://www.metatrader5.com/ru/trading-platform/web-trading> (дата обращения: 25.01.2026).
15. TradingView [Электронный ресурс]. — Режим доступа: <https://ru.tradingview.com/> (дата обращения: 25.01.2026).
16. Thinkorswim [Электронный ресурс]. — Режим доступа: <https://www.schwab.com/trading/thinkorswim/desktop> (дата обращения: 25.01.2026).
17. Bhatia J., Chaudhary K. The Definitive Guide to Google Vertex AI: Accelerate Your Machine Learning Journey with Google Cloud Vertex AI and MLOps Best Practices. – Packt Publishing Ltd, 2023.
18. Amazon Forecast [Электронный ресурс]. — Режим доступа: <https://aws.amazon.com/ru/forecast/> (дата обращения: 25.01.2026).
19. DataRobot [Электронный ресурс]. — Режим доступа: <https://www.datarobot.com/> (дата обращения: 25.01.2026).
20. Айжан А. ИНСТРУМЕНТЫ АНАЛИЗА ДАННЫХ И ПРИЛОЖЕНИЯ ДЛЯ АНАЛИЗА ДАННЫХ В ПЛАНИРОВАНИИ СПРОСА //Вестник науки. – 2025. – Т. 1. – №. 6 (87). – С. 1343-1356.
21. Free Statistics and Forecasting Software [Электронный ресурс]. — Режим доступа: https://www.wessa.net/rwasp_arimaforecasting.wasp (дата обращения: 25.01.2026).
22. Лопухин А. М. Обзор математических методов и моделей для прогнозирования рынка производных финансовых инструментов в аграрном секторе региональной экономики //Вестник Северо-Кавказского федерального университета. – 2026. – №. 6. – С. 120-130.
23. Баторова А. Б., Бычковская Е. Я. МЕТОДЫ ПРОГНОЗИРОВАНИЯ ВРЕМЕННЫХ РЯДОВ //ББК 60.6 я43 Е 22. – С. 53.
24. Грешилов А. А., Стакун В. А., Стакун А. А. Математические методы построения прогнозов //М.: Радио и связь. – 1997. – Т. 112. – С. 3.
25. Бажанов Н. Н. Экспоненциальное сглаживание как метод прогнозирования временных рядов //Ответственный редактор: Сукиасян АА, к. э. н., ст. преп. – 2014. – С. 194.
26. Олейник М. А. Экономико-математическое моделирование рыночных показателей на основе модели Хольта-Винтерса //ББК 65.32 Э 40. – 2019. – С. 282.

27. Shumway R. H., Stoffer D. S. ARIMA models //Time series analysis and its applications: with R examples. – Cham : Springer International Publishing, 2017. – С. 75-163.
28. Li Y. F., Cao H. Prediction for tourism flow based on LSTM neural network //Procedia Computer Science. – 2018. – Т. 129. – С. 277-283.
29. Козлов С. В., Седенков С. А. Анализ LSTM и GRU моделей для построения прогнозов временных рядов //International Journal of Open Information Technologies. – 2024. – Т. 12. – №. 7. – С. 43-50.
30. Копылова Я. А. Исследование методов оптимизации использования механизма внимания в архитектуре нейронных сетей //Парадигма. – 2025. – №. 5-2. – С. 191-199.
31. Сермягин К. А., Пылькин А. Н. Построение эффективной архитектуры для высоконагруженных и масштабируемых сервисов на fastapi: принципы и лучшие практики //Вестник Рязанского государственного радиотехнического университета. – 2025. – №. 92. – С. 49.
32. Лернер Р. Python: Pandas на практике. — М.: ДМК Пресс, 2021. — 256 с
33. Гончарова А. В., Смирнов С. Д. Парсинг и анализ данных, полученных с помощью API МОЕХ средствами Python //И ФИНАНСЫ. – С. 12.
34. Шакла Н. Машинное обучение и TensorFlow. – Питер, 2023.
35. Васяева Т. А., Мартыненко Т. В., Суббота Н. С. Прогнозирование финансовых временных рядов с помощью нейронных сетей с использованием библиотеки Keras в Python //Информатика и кибернетика. – 2019. – №. 2. – С. 41-50.
36. Мардан А. React быстро. Веб-приложения на React, JSX, Redux и GraphQL: Предисловие Джона Сонмеза. – Питер, 2023.
37. Recharts [Электронный ресурс]. — Режим доступа: <https://recharts.github.io/> (дата обращения: 25.01.2026).

А. Приложение А. Листинг программной реализации для прогноза котировок

Листинг А.1 – Python код для загрузки и обработки исторических данных

```
import time
import pandas as pd
import numpy as np
from datetime import datetime
ASSETS = {
```

```

"SBER": "Сбербанк",
"GAZP": "Газпром",
"LKOH": "Лукойл",
"ROSN": "Роснефть",
"NVTK": "Новатэк",
"TATN": "Татнефть"
}
def load_series(asset: str, start_date="2025-10-05", max_rows=10000):
    """Загрузка исторических данных с пагинацией"""
    if asset not in ASSETS: # ASSETS должен быть определен где-то в коде
        raise ValueError(f"Unknown asset: {asset}")
    url = f"https://iss.moex.com/iss/history/engines/stock/markets/shares/securities/{asset}.csv"
    # Параметры для первого запроса
    params = {
        'from': start_date,
        'till': datetime.now().strftime('%Y-%m-%d'),
        'start': 0
    }
    all_dataframes = []
    start_position = 0
    batch_size = 100 # MOEX ISS возвращает по 100 строк за запрос по умолчанию
    try:
        while True:
            # Формируем URL с текущей позицией пагинации
            current_url =
f"{url}?from={params['from']}&till={params['till']}&start={start_position}&marketprice_board=
1"

            # Загружаем данные
            df = pd.read_csv(
                current_url,
                sep=';',
                skiprows=2,
                encoding='windows-1251',
                skipfooter=4,
                engine='python'
            )
            # Если данные закончились
            if df.empty or len(df) == 0:
                break
            all_dataframes.append(df)

            # Если получили меньше batch_size строк, значит это последняя страница
            if len(df) < batch_size:
                break
            # Переходим к следующей порции данных
            start_position += len(df)
            # Проверяем лимит строк
            if start_position >= max_rows:
                print(f"Достигнут максимальный лимит строк: {max_rows}")
                break
            # Делаем небольшую паузу, чтобы не нагружать API
            time.sleep(0.1)
        # Объединяем все данные
        if not all_dataframes:

```

```

        return pd.Series()
    full_df = pd.concat(all_dataframes, ignore_index=True)
    # Обработываем данные
    full_df['TRADEDATE'] = pd.to_datetime(full_df['TRADEDATE'])
    full_df = full_df.sort_values('TRADEDATE')
    # Используем цену закрытия
    if 'CLOSE' in full_df.columns:
        price_column = 'CLOSE'
    else:
        price_column = 'LEGALCLOSEPRICE'
    series = full_df.set_index('TRADEDATE')[price_column]
    # Удаляем дубликаты и сортируем по дате
    series = series[~series.index.duplicated(keep='first')]
    series = series.sort_index()
    return series.dropna()
except Exception as e:
    print(f"Error loading {asset}: {e}")
    # Для отладки можно вывести информацию об ошибке
    # import traceback
    # traceback.print_exc()
    return pd.Series()

def generate_test_data(asset: str, start: str):
    """Генерирует тестовые данные если API недоступно"""
    dates = pd.date_range(start=start, end=datetime.now(), freq='D')
    n = len(dates)
    # Базовая цена в зависимости от акции
    base_prices = {
        "SBER": 300, "GAZP": 180, "LKOH": 7500,
        "ROSN": 600, "NVTK": 1700, "TATN": 350
    }
    base = base_prices.get(asset, 100)
    # Генерируем случайные данные с трендом
    np.random.seed(hash(asset) % 1000)
    noise = np.random.normal(0, 0.02, n).cumsum()
    trend = np.linspace(0, 0.5, n)
    prices = base * (1 + trend + noise)
    return pd.Series(prices, index=dates)

def data_split(series, validation_ratio=0.4, test_ratio=0.3):
    n = len(series)
    split1 = int(n * (1 - validation_ratio - test_ratio))
    split2 = int(n * (1 - test_ratio))
    train = series[:split1]
    val = series[split1:split2]
    test = series[split2:]
    return train, val, test

```

Листинг А.2 – Python код модели наивного прогноза

```

class NaiveForecaster(BaseForecaster):
    def forecast(self, full_series, train_end, val_end, steps, **kwargs):
        log_returns = np.log(full_series / full_series.shift(1)).dropna()
        test_start = val_end + pd.Timedelta(days=1)
        test_series = log_returns[test_start:]

```

```

errors_mae = []
errors_rmse = []
last_price = full_series.loc[val_end] # цена на конец валидации
# Для MASE считаем ошибки наивного прогноза "цена не изменится" (то же самое)
naive_errors = []
pred_prices = []
true_prices = []
for i, (date, true_logret) in enumerate(test_series.items()):
    # Прогноз доходности = 0
    pred_price = last_price * np.exp(0)
    pred_prices.append(pred_price)
    true_price = full_series.loc[date] # фактическая цена сегодня
    true_prices.append(true_price)
    # Ошибка
    mae = abs(true_price - pred_price)
    errors_mae.append(mae)
    errors_rmse.append(mae ** 2)
    naive_errors.append(abs(true_price - last_price))
    last_price = true_price
# Метрики
mae_val = np.mean(errors_mae)
rmse_val = np.sqrt(np.mean(errors_rmse))
smape_val = np.mean(2 * np.array(errors_mae) / (np.array([full_series.loc[date] for date in
test_series.index]) + last_price)) * 100 # упрощённо; в реальности лучше пошагово
mase_val = mae_val / np.mean(naive_errors) if np.mean(naive_errors) > 0 else 0
# Финальный прогноз от последней известной цены
final_last_price = full_series.iloc[-1]
pred = [final_last_price] * steps # цена не меняется
return {
    "pred": pred,
    "metrics": {"MAE": mae_val, "RMSE": rmse_val, "SMAPE": smape_val, "MASE":
mase_val},
    "info": {},
    "test_predictions": pred_prices,
    "test_actuals": true_prices,
    "test_naive_errors": list(naive_errors),
}

```

Листинг А.3 – Python код модели скользящего среднего

```

class MAModel:
    def __init__(self, history, window):
        self.history = history
        self.window = window
class MAForecaster(BaseForecaster):
    def forecast(self, full_series, train_end, val_end, steps, window_range=range(3, 365),
**kwargs):
    def ma_model_factory(history, params):
        if params is None:
            raise ValueError("params must be provided for training")
        window = params['window']
        return MAModel(history, window)
    def ma_predict_func(model, steps):
        # model типа MAModel

```



```

if len(model.history) < model.window:
    # если недостаточно данных, возвращаем нули
    return np.zeros(steps)
mean_ret = model.history.iloc[-model.window:].mean()
return np.array([mean_ret] * steps)
# Подбор окна на валидации
param_list = [{'window': w} for w in window_range]
best_params, _ = walk_forward_evaluate(
    full_series, train_end, val_end, steps,
    model_factory=ma_model_factory,
    predict_func=ma_predict_func,
    param_combinations=param_list,
    val_mode=True
)
# Тестирование с лучшим окном
# Для тестового режима фиксируем параметры в фабрике
def fixed_model_factory(history, params):
    # params игнорируется
    return MAModel(history, best_params['window'])
result = walk_forward_evaluate(
    full_series, train_end, val_end, steps,
    model_factory=fixed_model_factory,
    predict_func=ma_predict_func,
    param_combinations=None,
    val_mode=False
)
result['info'] = {'best_window': best_params['window']}
return result

```

Листинг А.4 – Python код модели экспоненциального сглаживания

```

class ExpSmoothingForecaster(BaseForecaster):
    def forecast(self, full_series, train_end, val_end, steps,
                 trend='add', # 'add', 'mul', None
                 damped_trend=True,
                 alpha_range=None, # если None – подбираем автоматически
                 **kwargs):
        if alpha_range is None:
            alpha_range = np.arange(0.1, 1.0, 0.1)
        def ets_model_factory(history, params):
            alpha = params['alpha']
            try:
                # Если trend=None, то damped_trend игнорируется
                model = ExponentialSmoothing(history, trend=trend, damped_trend=damped_trend if
trend else False)
                fit = model.fit(smoothing_level=alpha)
                return fit
            except:
                return None
        def predict_func(model, steps):
            return model.forecast(steps)
        param_list = [{'alpha': a} for a in alpha_range]
        best_params, _ = walk_forward_evaluate(
            full_series, train_end, val_end, steps,

```

```

        model_factory=ets_model_factory,
        predict_func=predict_func,
        param_combinations=param_list,
        val_mode=True
    )
    # Тестирование
    def fixed_factory(history, params):
        return ets_model_factory(history, best_params)
    result = walk_forward_evaluate(
        full_series, train_end, val_end, steps,
        model_factory=fixed_factory,
        predict_func=predict_func,
        param_combinations=None,
        val_mode=False
    )
    result['info'] = best_params
    return result

```

Листинг А.5 – Python код модели ARIMA

```

class AutoARIMAForecaster(BaseForecaster):
    def forecast(self, full_series, train_end, val_end, steps, **kwargs):
        # Параметры, управляющие поиском auto_arima (можно передать через kwargs)
        max_p = kwargs.get('max_p', 3)
        max_q = kwargs.get('max_q', 3)
        max_d = kwargs.get('max_d', 1)
        seasonal = kwargs.get('seasonal', False)
        stepwise = kwargs.get('stepwise', True)
        n_jobs = kwargs.get('n_jobs', 1)
        error_action = kwargs.get('error_action', 'ignore')
        suppress_warnings = kwargs.get('suppress_warnings', True)
        maxiter = kwargs.get('maxiter', 50)
        def arima_model_factory(history, params=None):
            try:
                model = auto_arima(
                    history,
                    start_p=0, start_q=0,
                    max_p=max_p, max_q=max_q, max_d=max_d,
                    seasonal=seasonal,
                    stepwise=stepwise,
                    n_jobs=n_jobs,
                    error_action=error_action,
                    suppress_warnings=suppress_warnings,
                    maxiter=maxiter
                )
                return model
            except Exception as e:
                print(f"AutoARIMA failed: {e}")
                return None
        def arima_predict_func(model, steps):
            return model.predict(n_periods=steps)
        # Для режима подбора на валидации передаём одну фиктивную комбинацию,
        # чтобы walk_forward_evaluate просто вызывал фабрику.
        best_params, _ = walk_forward_evaluate(
            full_series, train_end, val_end, steps,

```

```

    model_factory=arima_model_factory,
    predict_func=arima_predict_func,
    param_combinations=[{}], # одна пустая комбинация
    val_mode=True
)
# Для тестового режима фиксированная фабрика (та же самая, параметров нет)
def fixed_factory(history, params=None):
    return arima_model_factory(history)
result = walk_forward_evaluate(
    full_series, train_end, val_end, steps,
    model_factory=fixed_factory,
    predict_func=arima_predict_func,
    param_combinations=None,
    val_mode=False
)
# Информация о лучшем порядке уже содержится в последней модели,
result['info'] = {'method': 'auto_arima'}
return result

```

Листинг А.6 – Python код модели Prophet

```

class ProphetForecaster(BaseForecaster):
    def __init__(self, yearly_seasonality='auto', weekly_seasonality='auto',
daily_seasonality=False):
        self.yearly_seasonality = yearly_seasonality
        self.weekly_seasonality = weekly_seasonality
        self.daily_seasonality = daily_seasonality
    def forecast(self, full_series, train_end, val_end, steps, **kwargs):
        full_series = full_series.sort_index()
        log_returns = np.log(full_series / full_series.shift(1)).dropna()
        df = pd.DataFrame({
            'ds': log_returns.index,
            'y': log_returns.values
        })
        train_val = df[df['ds'] <= val_end].copy()
        model = Prophet(
            yearly_seasonality=self.yearly_seasonality,
            weekly_seasonality=self.weekly_seasonality,
            daily_seasonality=self.daily_seasonality
        )
        model.fit(train_val)
        test_dates = df[df['ds'] > val_end]['ds']
        future = pd.DataFrame({'ds': test_dates})
        forecast_test = model.predict(future)
        pred_logret = forecast_test['yhat'].values
        last_price = full_series.loc[val_end]
        pred_prices = last_price * np.exp(np.cumsum(pred_logret))
        true_prices = full_series.loc[test_dates].values
        naive_errors = np.abs(true_prices - last_price)
        mae = np.mean(np.abs(true_prices - pred_prices))
        rmse = np.sqrt(np.mean((true_prices - pred_prices) ** 2))
        smape = np.mean(2 * np.abs(true_prices - pred_prices) / (np.abs(true_prices) +
np.abs(pred_prices))) * 100
        mase = mae / np.mean(naive_errors) if np.mean(naive_errors) > 0 else np.nan
        future_steps = pd.date_range(start=full_series.index[-1] + pd.Timedelta(days=1),

```

```

periods=steps, freq='D')
future_final = pd.DataFrame({'ds': future_steps})
forecast_final = model.predict(future_final)
final_pred_logret = forecast_final['yhat'].values
final_pred_price = full_series.iloc[-1] * np.exp(np.cumsum(final_pred_logret))
return {
    "pred": list(final_pred_price),
    "metrics": {"MAE": mae, "RMSE": rmse, "SMAPE": smape, "MASE": mase},
    "info": {"method": "Prophet"},
    "test_predictions": list(pred_prices),
    "test_actuals": list(true_prices),
    "test_naive_errors": list(naive_errors),
}

```

Листинг А.7 – Python код модели LSTM

```

class LSTMForecaster(BaseForecaster):
    def __init__(self, window_size=20, hidden_units=50, epochs=20, batch_size=32):
        self.window_size = window_size
        self.hidden_units = hidden_units
        self.epochs = epochs
        self.batch_size = batch_size
    def _create_sequences(self, data):
        X, y = [], []
        for i in range(len(data) - self.window_size):
            X.append(data[i:i + self.window_size])
            y.append(data[i + self.window_size])
        X = np.array(X).reshape(-1, self.window_size, 1)
        y = np.array(y)
        return X, y
    def _build_model(self):
        model = Sequential([
            LSTM(self.hidden_units, input_shape=(self.window_size, 1)),
            Dense(1)
        ])
        model.compile(optimizer="adam", loss="mse")
        return model
    def _forecast_recursive(self, model, history, steps):
        seq = np.array(history[-self.window_size:], dtype=float)
        preds = []
        for _ in range(steps):
            x = seq.reshape(1, self.window_size, 1)
            pred = model.predict(x, verbose=0)[0, 0]
            preds.append(pred)
            seq = np.concatenate([seq[1:], [pred]])
        return np.array(preds)

    def forecast(self, full_series, train_end, val_end, steps, **kwargs):
        self.window_size = kwargs.get("window_size", self.window_size)
        self.hidden_units = kwargs.get("hidden_units", self.hidden_units)
        self.epochs = kwargs.get("epochs", self.epochs)
        self.batch_size = kwargs.get("batch_size", self.batch_size)
        full_series = full_series.sort_index()

```

```

log_returns = np.log(full_series / full_series.shift(1)).dropna()
test_series = log_returns[val_end + pd.Timedelta(days=1):]
history = log_returns[:val_end].copy()
last_price = full_series.loc[val_end]
pred_prices = []
true_prices = []
naive_errors = []
for i in range(len(test_series)):
    X_train, y_train = self._create_sequences(history.values)
    model = self._build_model()
    model.fit(
        X_train, y_train,
        epochs=self.epochs,
        batch_size=self.batch_size,
        verbose=0
    )
    pred_log = self._forecast_recursive(
        model,
        history.values,
        steps
    )

    assert len(pred_log) == steps, f"BUG: got {len(pred_log)} instead of {steps}"
    pred_price = last_price * np.exp(np.sum(pred_log))
    true_price = full_series.loc[test_series.index[i]]
    pred_prices.append(pred_price)
    true_prices.append(true_price)
    naive_errors.append(abs(true_price - last_price))
    history = pd.concat([
        history,
        pd.Series([test_series.iloc[i]], index=[test_series.index[i]])
    ])
    last_price = full_series.loc[test_series.index[i]]

pred_prices = np.array(pred_prices)
true_prices = np.array(true_prices)
mae = np.mean(np.abs(true_prices - pred_prices))
rmse = np.sqrt(np.mean((true_prices - pred_prices) ** 2))
smape = np.mean(
    2 * np.abs(true_prices - pred_prices) /
    (np.abs(true_prices) + np.abs(pred_prices))
) * 100
mase = mae / np.mean(naive_errors) if np.mean(naive_errors) > 0 else np.nan
return {
    "pred": list(pred_prices)[:steps],
    "metrics": {
        "MAE": mae,
        "RMSE": rmse,
        "SMAPE": smape,
        "MASE": mase
    },
    "info": {
        "method": "LSTM",
        "window_size": self.window_size,

```

```

        "hidden_units": self.hidden_units,
        "epochs": self.epochs
    },
    "test_predictions": list(pred_prices),
    "test_actuals": list(true_prices),
    "test_naive_errors": list(naive_errors),
}

```

Листинг А.8 – Python код модели GRU

```

class GRUForecaster(BaseForecaster):
    def __init__(self, window_size=20, hidden_units=50, epochs=20, batch_size=32):
        self.window_size = window_size
        self.hidden_units = hidden_units
        self.epochs = epochs
        self.batch_size = batch_size
    def _create_sequences(self, data):
        X, y = [], []
        for i in range(len(data) - self.window_size):
            X.append(data[i:i + self.window_size])
            y.append(data[i + self.window_size])
        return np.array(X).reshape(-1, self.window_size, 1), np.array(y)
    def _build_model(self):
        model = Sequential([
            GRU(self.hidden_units, input_shape=(self.window_size, 1)),
            Dense(1)
        ])
        model.compile(optimizer="adam", loss="mse")
        return model
    def _forecast_recursive(self, model, history, steps):
        seq = history[-self.window_size:].copy()
        preds = []
        for _ in range(steps):
            x = seq.reshape(1, self.window_size, 1)
            pred = model.predict(x, verbose=0)[0, 0]
            preds.append(pred)
            seq = np.append(seq[1:], pred)
        return np.array(preds)

    def forecast(self, full_series, train_end, val_end, steps, **kwargs):
        self.window_size = kwargs.get("window_size", self.window_size)
        self.hidden_units = kwargs.get("hidden_units", self.hidden_units)
        self.epochs = kwargs.get("epochs", self.epochs)
        self.batch_size = kwargs.get("batch_size", self.batch_size)
        full_series = full_series.sort_index()
        log_returns = np.log(full_series / full_series.shift(1)).dropna()
        train_res = log_returns[:val_end]
        test_res = log_returns[val_end + pd.Timedelta(days=1):]
        history = train_res.copy()
        last_price = full_series.loc[val_end]
        pred_prices = []
        true_prices = []
        naive_errors = []
        for i in range(len(test_res)):

```

```

X_train, y_train = self._create_sequences(history.values)

model = self._build_model()

es = EarlyStopping(monitor="loss", patience=5, restore_best_weights=True)

model.fit(
    X_train, y_train,
    epochs=self.epochs,
    batch_size=self.batch_size,
    verbose=0,
    callbacks=[es]
)

pred_logret = self._forecast_recursive(model, history.values, steps)
pred_price = last_price * np.exp(pred_logret[0]) # берём первый шаг

true_price = full_series.loc[test_res.index[i]]

pred_prices.append(pred_price)
true_prices.append(true_price)
naive_errors.append(abs(true_price - last_price))

# обновляем историю
history = pd.concat([history, pd.Series([test_res.iloc[i]], index=[test_res.index[i]])])
last_price = full_series.loc[test_res.index[i]]
pred_prices = np.array(pred_prices)
true_prices = np.array(true_prices)
mae = np.mean(np.abs(true_prices - pred_prices))
rmse = np.sqrt(np.mean((true_prices - pred_prices)**2))
smape = np.mean(2 * np.abs(true_prices - pred_prices) / (np.abs(true_prices) +
np.abs(pred_prices))) * 100
mase = mae / np.mean(naive_errors) if np.mean(naive_errors) > 0 else np.nan
return {
    "pred": list(pred_prices)[:steps],
    "metrics": {"MAE": mae, "RMSE": rmse, "SMAPE": smape, "MASE": mase},
    "info": {"method": "GRU", "window_size": self.window_size, "hidden_units":
self.hidden_units},
    "test_predictions": list(pred_prices),
    "test_actuals": list(true_prices),
    "test_naive_errors": list(naive_errors),
}

```

Листинг А.9 – Python код модели transformer

```

def transformer_encoder(x, d_model, num_heads, ff_dim, dropout_rate=0.1):
    attn_output = MultiHeadAttention(
        num_heads=num_heads, key_dim=d_model // num_heads
    )(x, x)
    attn_output = Dropout(dropout_rate)(attn_output)
    out1 = LayerNormalization(epsilon=1e-6)(x + attn_output)
    ffn = Dense(ff_dim, activation='relu')(out1)
    ffn = Dense(d_model)(ffn)
    ffn = Dropout(dropout_rate)(ffn)
    out2 = LayerNormalization(epsilon=1e-6)(out1 + ffn)

```

```

    return out2
def positional_encoding(length, depth):
    positions = np.arange(length)[: , np.newaxis]
    depths = np.arange(depth)[np.newaxis, :]
    angle_rates = 1 / np.power(10000, (2 * (depths // 2)) / np.float32(depth))
    angle_rads = positions * angle_rates
    # sin для чётных индексов, cos для нечётных
    angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])
    angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2])
    pos_enc = angle_rads[np.newaxis, ...] # (1, length, depth)
    return tf.constant(pos_enc, dtype=tf.float32)

class TransformerForecaster(BaseForecaster):
    def __init__(
        self,
        window_size=20,
        d_model=64,
        num_heads=4,
        num_layers=2,
        ff_dim=128,
        dropout_rate=0.1,
        epochs=20,
        batch_size=32
    ):
        self.window_size = window_size
        self.d_model = d_model
        self.num_heads = num_heads
        self.num_layers = num_layers
        self.ff_dim = ff_dim
        self.dropout_rate = dropout_rate
        self.epochs = epochs
        self.batch_size = batch_size

    def _create_sequences(self, data):
        X, y = [], []
        for i in range(len(data) - self.window_size):
            X.append(data[i:i + self.window_size])
            y.append(data[i + self.window_size])
        X = np.array(X).reshape(-1, self.window_size, 1)
        y = np.array(y)
        return X, y

    def _build_model(self):
        input_seq = Input(shape=(self.window_size, 1))
        x = Dense(self.d_model)(input_seq) # (batch, window, d_model)
        pos_enc = positional_encoding(self.window_size, self.d_model)
        x = Lambda(lambda z: z + pos_enc)(x)
        for _ in range(self.num_layers):
            x = transformer_encoder(
                x, self.d_model, self.num_heads,
                self.ff_dim, self.dropout_rate
            )
        x = Lambda(lambda z: z[:, -1, :])(x) # (batch, d_model)
        output = Dense(1)(x) # (batch, 1)

```



```

    model = Model(inputs=input_seq, outputs=output)
    model.compile(optimizer='adam', loss='mse')
    return model
def _forecast_recursive(self, model, history, steps):
    seq = np.array(history[-self.window_size:], dtype=float)
    preds = []
    for _ in range(steps):
        x = seq.reshape(1, self.window_size, 1)
        pred = model.predict(x, verbose=0)[0, 0]
        preds.append(pred)
        seq = np.concatenate([seq[1:], [pred]])
    return np.array(preds) # ГАРАНТИЯ РАЗМЕРА steps

def forecast(self, full_series, train_end, val_end, steps, **kwargs):
    self.window_size = kwargs.get('window_size', self.window_size)
    self.d_model = kwargs.get('d_model', self.d_model)
    self.num_heads = kwargs.get('num_heads', self.num_heads)
    self.num_layers = kwargs.get('num_layers', self.num_layers)
    self.ff_dim = kwargs.get('ff_dim', self.ff_dim)
    self.dropout_rate = kwargs.get('dropout_rate', self.dropout_rate)
    self.epochs = kwargs.get('epochs', self.epochs)
    self.batch_size = kwargs.get('batch_size', self.batch_size)
    full_series = full_series.sort_index()
    log_returns = np.log(full_series / full_series.shift(1)).dropna()
    test_series = log_returns[val_end + pd.Timedelta(days=1):]
    history = log_returns[:val_end].copy()
    last_price = full_series.loc[val_end]
    pred_prices = []
    true_prices = []
    naive_errors = []
    for i in range(len(test_series)):
        X_train, y_train = self._create_sequences(history.values)

        model = self._build_model()
        model.fit(
            X_train, y_train,
            epochs=self.epochs,
            batch_size=self.batch_size,
            verbose=0
        )

        pred_log = self._forecast_recursive(
            model,
            history.values,
            steps
        )

        assert len(pred_log) == steps, \
            f'BUG: got {len(pred_log)} instead of {steps}'
        pred_price = last_price * np.exp(np.sum(pred_log))
        true_price = full_series.loc[test_series.index[i]]

        pred_prices.append(pred_price)

```

```

true_prices.append(true_price)
naive_errors.append(abs(true_price - last_price))
history = pd.concat([
    history,
    pd.Series([test_series.iloc[i]], index=[test_series.index[i]])
])

last_price = full_series.loc[test_series.index[i]]
pred_prices = np.array(pred_prices)
true_prices = np.array(true_prices)
mae = np.mean(np.abs(true_prices - pred_prices))
rmse = np.sqrt(np.mean((true_prices - pred_prices) ** 2))
smape = np.mean(
    2 * np.abs(true_prices - pred_prices) /
    (np.abs(true_prices) + np.abs(pred_prices))
) * 100

mase = mae / np.mean(naive_errors) if np.mean(naive_errors) > 0 else np.nan

return {
    'pred': list(pred_prices)[:steps],
    'metrics': {
        'MAE': mae,
        'RMSE': rmse,
        'SMAPE': smape,
        'MASE': mase
    },
    'info': {
        'method': 'Transformer',
        'window_size': self.window_size,
        'd_model': self.d_model,
        'num_heads': self.num_heads,
        'num_layers': self.num_layers,
        'epochs': self.epochs
    },
    'test_predictions': list(pred_prices),
    'test_actuals': list(true_prices),
    'test_naive_errors': list(naive_errors),
}

```

Листинг А.10 – JavaScript-код веб приложения.

```

import React, { useEffect, useState, useMemo } from "react";
import {
    LineChart, Line, XAxis, YAxis, Tooltip,
    CartesianGrid, Legend, ResponsiveContainer, Area, ComposedChart
} from "recharts";
import {
    TextField, Button, MenuItem, Select,
    InputLabel, FormControl, Box, Typography,
    Card, CardContent, Grid, Chip, Paper,
    alpha, useTheme, Divider,
    Accordion, AccordionSummary, AccordionDetails,

```

```

    Checkbox, FormControlLabel, Stack, Slider
  } from "@mui/material";
import {
  TrendingUp, ShowChart, Assessment,
  Timeline, BarChart, Refresh, DateRange,
  ExpandMore, Tune, PieChart
} from "@mui/icons-material";

const API_BASE = "http://localhost:8000";

/* ===== Гиперпараметры ===== */
const modelParamsMeta = {
  naive: [
    { name: "seasonal_period", label: "Период сезонности", type: "number", default: 1, min: 1,
    step: 1,
      helperText: "Для Seasonal Naive (0 — без сезонности)" }
  ],
  ma: [
    { name: "window", label: "Размер окна", type: "number", default: 5, min: 1, step: 1 }
  ],
  exp: [
    { name: "alpha", label: " $\alpha$  (сглаживание уровня)", type: "number", default: 0.2, min: 0, max: 1,
    step: 0.05 },
    { name: "beta", label: " $\beta$  (тренд)", type: "number", default: 0.1, min: 0, max: 1, step: 0.05 },
    { name: "gamma", label: " $\gamma$  (сезонность)", type: "number", default: 0.1, min: 0, max: 1, step:
    0.05 },
    { name: "seasonal_periods", label: "Период сезонности", type: "number", default: 7, min: 0,
    step: 1 },
    { name: "trend", label: "Учитывать тренд", type: "checkbox", default: false },
    { name: "seasonal", label: "Учитывать сезонность", type: "checkbox", default: false },
    { name: "model_type", label: "Тип модели", type: "select", default: "simple",
      options: ["simple", "holt", "holt-winters"] }
  ],
  arima: [
    { name: "p", label: "p (AR)", type: "number", default: 0, min: 0, step: 1 },
    { name: "d", label: "d (I)", type: "number", default: 0, min: 0, step: 1 },
    { name: "q", label: "q (MA)", type: "number", default: 0, min: 0, step: 1 },
  ],
  prophet: [
    { name: "changepoint_prior_scale", label: "changepoint_prior_scale", type: "number", default:
    0.05, min: 0.001, max: 1, step: 0.01 },
    { name: "seasonality_prior_scale", label: "seasonality_prior_scale", type: "number", default: 10,
    min: 0.1, max: 100, step: 0.1 },
    { name: "yearly_seasonality", label: "Годовая сезонность", type: "checkbox", default: true },
    { name: "weekly_seasonality", label: "Недельная сезонность", type: "checkbox", default: true
    },
    { name: "daily_seasonality", label: "Дневная сезонность", type: "checkbox", default: false }
  ],
  lstm: [
    { name: "units", label: "Число нейронов", type: "number", default: 50, min: 1, step: 1 },
    { name: "epochs", label: "Эпохи", type: "number", default: 20, min: 1, step: 1 },
    { name: "batch_size", label: "Размер батча", type: "number", default: 32, min: 1, step: 1 },
    { name: "dropout", label: "Dropout", type: "number", default: 0.2, min: 0, max: 0.9, step: 0.05
    },
  ],

```

```

    { name: "learning_rate", label: "Скорость обучения", type: "number", default: 0.001, min: 0.0001, max: 0.1, step: 0.0001, format: "exponential" },
    { name: "lookback", label: "Окно истории", type: "number", default: 10, min: 1, step: 1 }
  ],
  gru: [
    { name: "units", label: "Число нейронов", type: "number", default: 50, min: 1, step: 1 },
    { name: "epochs", label: "Эпохи", type: "number", default: 20, min: 1, step: 1 },
    { name: "batch_size", label: "Размер батча", type: "number", default: 32, min: 1, step: 1 },
    { name: "dropout", label: "Dropout", type: "number", default: 0.2, min: 0, max: 0.9, step: 0.05 }
  ],
  { name: "learning_rate", label: "Скорость обучения", type: "number", default: 0.001, min: 0.0001, max: 0.1, step: 0.0001, format: "exponential" },
  { name: "lookback", label: "Окно истории", type: "number", default: 10, min: 1, step: 1 }
],
transformer: [
  { name: "units", label: "Число нейронов", type: "number", default: 50, min: 1, step: 1 },
  { name: "epochs", label: "Эпохи", type: "number", default: 20, min: 1, step: 1 },
  { name: "batch_size", label: "Размер батча", type: "number", default: 32, min: 1, step: 1 },
  { name: "dropout", label: "Dropout", type: "number", default: 0.2, min: 0, max: 0.9, step: 0.05 }
],
  { name: "learning_rate", label: "Скорость обучения", type: "number", default: 0.001, min: 0.0001, max: 0.1, step: 0.0001, format: "exponential" },
  { name: "lookback", label: "Окно истории", type: "number", default: 10, min: 1, step: 1 }
]
};

```

```

const getDefaultParams = (model) => {
  const meta = modelParamsMeta[model];
  if (!meta) return {};
  const defaults = {};
  meta.forEach(field => {
    defaults[field.name] = field.default;
  });
  return defaults;
};

```

```

export default function App() {
  const [assets, setAssets] = useState([]);
  const [selectedAsset, setSelectedAsset] = useState("SBER");
  const [model1, setModel1] = useState("arima");
  const [model2, setModel2] = useState("");
  const [steps, setSteps] = useState(20);
  const [startDate, setStartDate] = useState("2025-10-01");
  const [data, setData] = useState([]);
  const [metrics, setMetrics] = useState(null);
  const [loading, setLoading] = useState(false);

  const [hyper1, setHyper1] = useState(getDefaultParams("arima"));
  const [hyper2, setHyper2] = useState({});
  const [auto1, setAuto1] = useState(false);
  const [auto2, setAuto2] = useState(false);
  const [ensembleWeight, setEnsembleWeight] = useState(0.5);

```

```

// Состояние для пропорций разбиения (в процентах)

```

```
const [splitRatios, setSplitRatios] = useState({
  train: 60,
  val: 20,
  test: 20
});
```

```
const theme = useTheme();
```

```
// Валидация: сумма должна быть 100%
const splitValid = useMemo(() => {
  const sum = splitRatios.train + splitRatios.val + splitRatios.test;
  return Math.abs(sum - 100) < 0.01;
}, [splitRatios]);
```

```
// Обновление одного из полей разбиения
const handleSplitChange = (field, value) => {
  setSplitRatios(prev => ({ ...prev, [field]: value }));
};
```

```
useEffect(() => {
  setHyper1(getDefaultParams(model1));
  setAuto1(false);
}, [model1]);
```

```
useEffect(() => {
  if (model2) {
    setHyper2(getDefaultParams(model2));
    setAuto2(false);
  } else {
    setHyper2({});
  }
}, [model2]);
```

```
useEffect(() => {
  fetch(`${API_BASE}/assets`)
    .then((res) => res.json())
    .then((data) => setAssets(Object.keys(data)));
}, []);
```

```
const buildSingleHyperQuery = (hyper, auto) => {
  if (auto) return "autotune=true";
  const parts = [];
  for (const [key, value] of Object.entries(hyper)) {
    parts.push(`${encodeURIComponent(key)}=${encodeURIComponent(value)}`);
  }
  return parts.join("&");
};
```

```
const buildEnsembleHyperQuery = (modelKey, hyper, auto) => {
  if (auto) return `${modelKey}_autotune=true`;
  const parts = [];
  for (const [key, value] of Object.entries(hyper)) {
```

```
parts.push(`${modelKey}_${encodeURIComponent(key)}=${encodeURIComponent(value)}`);
```

```

    }
    return parts.join("&");
};

const fetchData = async () => {
    if (!selectedAsset || !model1 || !splitValid) return;
    setLoading(true);

    try {
        // Переводим проценты в доли
        const trainRatio = splitRatios.train / 100;
        const valRatio = splitRatios.val / 100;
        const testRatio = splitRatios.test / 100;

        const baseParams =
`asset=${selectedAsset}&steps=${steps}&start_date=${startDate}&train_ratio=${trainRatio}&val_ratio=${valRatio}&test_ratio=${testRatio}`;
        let url = "";

        if (model2) {
            const hyper1Query = buildEnsembleHyperQuery("model1", hyper1, auto1);
            const hyper2Query = buildEnsembleHyperQuery("model2", hyper2, auto2);
            url =
`${API_BASE}/forecast?${baseParams}&model1=${model1}&model2=${model2}&ensemble_weight=${ensembleWeight}`;
        } else {
            const hyper1Query = buildSingleHyperQuery(hyper1, auto1);
            url = `${API_BASE}/forecast?${baseParams}&model1=${model1}`;
        }

        const response = await fetch(url);
        const forecastData = await response.json();

        const chartData = [];
        const allDates = forecastData.dates;
        const trainLen = forecastData.train.length;
        const valLen = forecastData.val.length;
        const testLen = forecastData.test.length;
        const forecastLen = forecastData.forecast.length;

        for (let i = 0; i < trainLen + valLen + testLen; i++) {
            let actual = null;
            if (i < trainLen) actual = forecastData.train[i];
            else if (i < trainLen + valLen) actual = forecastData.val[i - trainLen];
            else actual = forecastData.test[i - trainLen - valLen];
            chartData.push({ date: allDates[i], actual, predicted: null });
        }

        const lastActualIdx = trainLen + valLen + testLen - 1;
        chartData.push({
            date: allDates[lastActualIdx],
            actual: chartData[lastActualIdx].actual,
            predicted: chartData[lastActualIdx].actual
        });
    }
};

```

```

for (let i = 0; i < forecastLen; i++) {
  chartData.push({
    date: allDates[lastActualIdx + 1 + i],
    actual: null,
    predicted: forecastData.forecast[i]
  });
}

setData(chartData);
setMetrics(forecastData.metrics);
} finally {
  setLoading(false);
}
};

const renderHyperFields = (model, hyper, setHyper, auto, setAuto) => {
  const meta = modelParamsMeta[model];
  if (!meta) return null;

  const handleChange = (name, value) => {
    setHyper(prev => ({ ...prev, [name]: value }));
  };

  return (
    <Box>
      <FormControlLabel
        control=<Checkbox checked={auto} onChange={(e) => setAuto(e.target.checked)} />
        label="Автоматический подбор гиперпараметров"
      />
      <Stack spacing={2} sx={{ mt: 1 }}>
        {meta.map((field) => {
          const disabled = auto;
          const value = hyper[field.name] ?? field.default;
          switch (field.type) {
            case "number":
              return (
                <TextField
                  key={field.name}
                  label={field.label}
                  type="number"
                  value={value}
                  onChange={(e) => handleChange(field.name, parseFloat(e.target.value) || 0)}
                  disabled={disabled}
                  fullWidth
                  variant="outlined"
                  inputProps={{ min: field.min, max: field.max, step: field.step }}
                  helperText={field.helperText}
                  size="small"
                />
              );
            case "checkbox":
              return (
                <FormControlLabel

```

```

        key={field.name}
        control={
          <Checkbox
            checked={!value}
            onChange={(e) => handleChange(field.name, e.target.checked)}
            disabled={disabled}
          />
        }
        label={field.label}
      />
    );
    case "select":
      return (
        <FormControl fullWidth size="small" key={field.name} disabled={disabled}>
          <InputLabel>{field.label}</InputLabel>
          <Select
            value={value}
            label={field.label}
            onChange={(e) => handleChange(field.name, e.target.value)}
          >
            {field.options.map(opt => (
              <MenuItem key={opt} value={opt}>{opt}</MenuItem>
            ))}
          </Select>
        </FormControl>
      );
    default:
      return null;
  }
  }}}
</Stack>
</Box>
);
};

return (
  <Box sx={{
    minHeight: "100vh",
    background: `linear-gradient(135deg, ${alpha(theme.palette.primary.dark, 0.05)} 0%,
    ${alpha(theme.palette.secondary.dark, 0.02)} 100%)`,
    position: "relative", overflow: "hidden",
    "&::before": {
      content: "''", position: "absolute", top: 0, left: 0, right: 0, bottom: 0,
      background: "radial-gradient(circle at 20% 50%, rgba(33, 150, 243, 0.08) 0%, transparent
    50%)",
      pointerEvents: "none"
    }
  }}>
    <Box sx={{ p: 4, position: "relative", zIndex: 1 }}>
      { /* Header */ }
      <Paper elevation={0} sx={{ mb: 4, p: 3, background: `linear-gradient(135deg,
      ${theme.palette.primary.main} 0%, ${theme.palette.primary.dark} 100%)`, borderRadius: 4,
      boxShadow: `0 10px 40px ${alpha(theme.palette.primary.main, 0.3)}`, position: "relative",
      overflow: "hidden" }}>

```



```

    <Box sx={{ position: "relative", zIndex: 1 }}>
      <Typography variant="h4" fontWeight="bold" sx={{ color: "white", display: "flex",
alignItems: "center", gap: 1, mb: 1 }}>
        <TrendingUp sx={{ fontSize: 35 }} />
        ИАС для моделирования и прогнозирования биржевых котировок
      </Typography>
      <Typography variant="body1" sx={{ color: alpha("#fff", 0.9), ml: 6 }}>
        Интеллектуальная система анализа и прогнозирования (одиночная модель или
ансамбль)
      </Typography>
    </Box>
    <Box sx={{ position: "absolute", top: -20, right: -20, opacity: 0.1, transform:
"rotate(15deg)" }}>
      <ShowChart sx={{ fontSize: 200, color: "white" }} />
    </Box>
  </Paper>

  { /* Controls */ }
  <Card elevation={0} sx={{ mb: 4, borderRadius: 4, boxShadow: `0 4px 20px
${alpha(theme.palette.common.black, 0.08)}`, border: `1px solid ${alpha(theme.palette.divider,
0.1)}`, backdropFilter: "blur(10px)", background: alpha(theme.palette.background.paper, 0.9) }}>
    <CardContent sx={{ p: 3 }}>
      <Grid container spacing={3} alignItems="center">
        <Grid item xs={12} md={2.5}>
          <FormControl fullWidth variant="outlined">
            <InputLabel>Актив</InputLabel>
            <Select value={selectedAsset} onChange={(e) => setSelectedAsset(e.target.value)}
label="Актив" sx={{ borderRadius: 3 }}>
              {assets.map((asset) => (
                <MenuItem key={asset} value={asset}>
                  <Box sx={{ display: "flex", alignItems: "center", gap: 1 }}>
                    <BarChart sx={{ fontSize: 18, color: theme.palette.primary.main }} />
                    {asset}
                  </Box>
                </MenuItem>
              ))}
            </Select>
          </FormControl>
        </Grid>
        <Grid item xs={12} md={2.5}>
          <FormControl fullWidth variant="outlined">
            <InputLabel>Модель 1</InputLabel>
            <Select value={model1} onChange={(e) => setModel1(e.target.value)}
label="Модель 1" sx={{ borderRadius: 3 }}>
              {Object.keys(modelParamsMeta).map(m => (
                <MenuItem key={m} value={m}>{m.toUpperCase()}</MenuItem>
              ))}
            </Select>
          </FormControl>
        </Grid>
        <Grid item xs={12} md={2.5}>
          <FormControl fullWidth variant="outlined">
            <InputLabel shrink>Модель 2 (ансамбль)</InputLabel>
            <Select

```

```

        value={model2}
        onChange={(e) => setModel2(e.target.value)}
        displayEmpty
        renderValue={(selected) => {
          if (!selected) {
            return <em style={{ color: alpha(theme.palette.text.primary, 0.4) }}>Не
выбрана</em>;
          }
          return selected.toUpperCase();
        }}
        sx={{ borderRadius: 3 }}
      >
      <MenuItem value="" disabled>
        <em>— Не выбрана —</em>
      </MenuItem>
      {Object.keys(modelParamsMeta).map(m => (
        <MenuItem key={m} value={m}>{m.toUpperCase()}</MenuItem>
      ))}
    </Select>
  </FormControl>
</Grid>
<Grid item xs={12} md={1.5}>
  <TextField type="number" label="Шаров" value={steps} onChange={(e) =>
setSteps(Number(e.target.value))} fullWidth variant="outlined" sx={{ "& .MuiOutlinedInput-
root": { borderRadius: 3 } }} />
</Grid>
<Grid item xs={12} md={2}>
  <TextField label="Начальная дата" type="date" value={startDate} onChange={(e) =>
setStartDate(e.target.value)} fullWidth variant="outlined" InputLabelProps={{ shrink: true }}
sx={{ "& .MuiOutlinedInput-root": { borderRadius: 3 } }} />
</Grid>
<Grid item xs={12} md={1}>
  <Button variant="contained" fullWidth onClick={fetchData} disabled={loading ||
!splitValid}
    startIcon={loading ? <Refresh className="spin" /> : <Timeline />}
    sx={{ height: 56, borderRadius: 3, fontWeight: "bold", background: `linear-
gradient(135deg, ${theme.palette.primary.main} 0%, ${theme.palette.primary.dark} 100%)`,
    boxShadow: `0 4px 15px ${alpha(theme.palette.primary.main, 0.4)}`, "&:hover": { transform:
"translateY(-2px)", boxShadow: `0 6px 20px ${alpha(theme.palette.primary.main, 0.5)}` },
    "&:active": { transform: "translateY(0)" } }}>
    {loading ? "Загрузка..." : "Прогноз"}
  </Button>
</Grid>
</Grid>

  {/* Блок разбиения данных */}
  <Box sx={{ mt: 3 }}>
    <Typography variant="subtitle1" sx={{ display: "flex", alignItems: "center", gap: 1, mb:
1 }}>
      <PieChart color="primary" fontSize="small" />
      Разбиение данных (обучающая / валидационная / тестовая)
    </Typography>
    <Grid container spacing={2}>
      <Grid item xs={4}>

```

```

<TextField
  label="Обучающая, %"
  type="number"
  value={splitRatios.train}
  onChange={(e) => handleSplitChange("train", Number(e.target.value))}
  fullWidth
  variant="outlined"
  size="small"
  inputProps={{ min: 0, max: 100, step: 1 }}
  error={!splitValid}
  helperText={!splitValid ? "Сумма должна быть 100%" : ""}
/>
</Grid>
<Grid item xs={4}>
  <TextField
    label="Валидационная, %"
    type="number"
    value={splitRatios.val}
    onChange={(e) => handleSplitChange("val", Number(e.target.value))}
    fullWidth
    variant="outlined"
    size="small"
    inputProps={{ min: 0, max: 100, step: 1 }}
    error={!splitValid}
  />
</Grid>
<Grid item xs={4}>
  <TextField
    label="Тестовая, %"
    type="number"
    value={splitRatios.test}
    onChange={(e) => handleSplitChange("test", Number(e.target.value))}
    fullWidth
    variant="outlined"
    size="small"
    inputProps={{ min: 0, max: 100, step: 1 }}
    error={!splitValid}
  />
</Grid>
</Grid>
</Box>

```

```

{/* Панель гиперпараметров */}
<Accordion sx={{ mt: 2, boxShadow: 0, border: `1px solid ${alpha(theme.palette.divider,
0.2)}`, borderRadius: 3 }}>
  <AccordionSummary expandIcon={<ExpandMore />}>
    <Box sx={{ display: "flex", alignItems: "center", gap: 1 }}>
      <Text color="primary" />
      <Typography fontWeight="bold">
        {model2 ? "Гиперпараметры ансамбля" : "Гиперпараметры модели"}
      </Typography>
    </Box>
  </AccordionSummary>
  <AccordionDetails>

```

```

<Grid container spacing={4}>
  <Grid item xs={12} md={model2 ? 6 : 12}>
    <Typography variant="subtitle1" gutterBottom>
      {model1.toUpperCase()}
    </Typography>
    {renderHyperFields(model1, hyper1, setHyper1, auto1, setAuto1)}
  </Grid>
  {model2 && (
    <Grid item xs={12} md={6}>
      <Typography variant="subtitle1" gutterBottom>
        {model2.toUpperCase()}
      </Typography>
      {renderHyperFields(model2, hyper2, setHyper2, auto2, setAuto2)}
    </Grid>
  )}
</Grid>

```

```

{model2 && (
  <Box sx={{ mt: 3, px: 1 }}>
    <Typography variant="subtitle1" gutterBottom>
      Вес модели 1 в ансамбле: {ensembleWeight.toFixed(2)}
    </Typography>
    <Slider
      value={ensembleWeight}
      onChange={(e, newValue) => setEnsembleWeight(newValue)}
      min={0}
      max={1}
      step={0.05}
      marks={[
        { value: 0, label: '0' },
        { value: 0.5, label: '0.5' },
        { value: 1, label: '1' }
      ]}
      valueLabelDisplay="auto"
      sx={{ maxWidth: 400 }}
    />
    <Typography variant="caption" color="text.secondary">
      Итоговый прогноз = вес * Прогноз1 + (1 - вес) * Прогноз2
    </Typography>
  </Box>
)}
</AccordionDetails>
</Accordion>

```

```

{/* Chips */}
<Box sx={{ mt: 3, display: "flex", gap: 1, flexWrap: "wrap" }}>
  <Chip icon={ Assessment } label={ `Актив: ${selectedAsset}` } variant="outlined"
  sx={{ borderRadius: 2 }} />
  <Chip icon={ Timeline } label={ `Модель: ${model1.toUpperCase()}${model2 ? " + " +
  + model2.toUpperCase() : ""}` } variant="outlined" sx={{ borderRadius: 2 }} />
  <Chip icon={ DateRange } label={ `Прогноз: ${steps} дн.` } variant="outlined" sx={{
  borderRadius: 2 }} />
  {model2 && <Chip label={ `Вес: ${ensembleWeight.toFixed(2)}` } variant="outlined"
  sx={{ borderRadius: 2 }} />}

```

```

    </Box>
  </CardContent>
</Card>

{/* Метрики */}
{metrics && (
  <>
    <Typography variant="h6" sx={{ mb: 2, display: "flex", alignItems: "center", gap: 1 }}>
      <Assessment /> Метрики {model2 ? "ансамбля" : `модели ${model1.toUpperCase()}`}
    </Typography>
    <Grid container spacing={3} sx={{ mb: 4 }}>
      {[
        { key: "RMSE", value: metrics.RMSE, icon: " 📊 ", color: "#FF6B6B", desc:
"Среднеквадратичная ошибка" },
        { key: "MAE", value: metrics.MAE, icon: " 📈 ", color: "#4ECDC4", desc: "Средняя
абсолютная ошибка" },
        { key: "MASE", value: metrics.MASE, icon: " 📉 ", color: "#45B7D1", desc: "Средняя
абсолютная масштабированная ошибка" },
        { key: "SMAPE", value: metrics.SMAPE.toFixed(4) + "%", icon: " 🎯 ", color:
"#6C5CE7", desc: "Симметричная средняя абсолютная процентная ошибка" }
      ].map((m) => (
        <Grid item xs={12} md={3} key={m.key}>
          <Card elevation={0} sx={{
            borderRadius: 4, boxShadow: `0 4px 20px ${alpha(m.color, 0.15)}`,
            border: `1px solid ${alpha(m.color, 0.2)}`,
            transition: "all 0.3s", "&:hover": { transform: "translateY(-4px)", boxShadow: `0 8px
30px ${alpha(m.color, 0.25)}` }
          }}>
            <CardContent sx={{ p: 3 }}>
              <Box sx={{ display: "flex", alignItems: "center", gap: 2, mb: 2 }}>
                <Box sx={{ width: 48, height: 48, borderRadius: 3, background: `linear-
gradient(135deg, ${alpha(m.color, 0.2)} 0%, ${alpha(m.color, 0.1)} 100%)`, display: "flex",
                alignItems: "center", justifyContent: "center", fontSize: 24 }}>
                  {m.icon}
                </Box>
                <Box>
                  <Typography variant="h6" fontWeight="bold">{m.key}</Typography>
                  <Typography variant="caption" color="text.secondary">{m.desc}</Typography>
                </Box>
              </Box>
              <Typography variant="h4" fontWeight="bold" sx={{ color: m.color, textAlign:
"right" }}>
                {typeof m.value === "number" ? m.value.toFixed(4) : m.value}
              </Typography>
            </CardContent>
          </Card>
        </Grid>
      )})
    </Grid>
  </>
)}
{/* График */}

```

```

<Card elevation={0} sx={{ borderRadius: 4, boxShadow: `0 4px 20px
${alpha(theme.palette.common.black, 0.08)}`, border: `1px solid ${alpha(theme.palette.divider,
0.1)}`, background: alpha(theme.palette.background.paper, 0.95), backdropFilter: "blur(10px)"
}}>
  <CardContent sx={{ p: 3 }}>
    <Box sx={{ display: "flex", alignItems: "center", gap: 1, mb: 3 }}>
      <ShowChart sx={{ color: theme.palette.primary.main }} />
      <Typography variant="h6" fontWeight="bold">
        График прогноза {model2 ? "ансамбля" : ""}
      </Typography>
      <Box sx={{ flex: 1 }} />
      <Box sx={{ display: "flex", gap: 2 }}>
        <Box sx={{ display: "flex", alignItems: "center", gap: 1 }}>
          <Box sx={{ width: 20, height: 3, bgcolor: "#3f51b5", borderRadius: 1 }} />
          <Typography variant="caption">Фактические данные</Typography>
        </Box>
        <Box sx={{ display: "flex", alignItems: "center", gap: 1 }}>
          <Box sx={{ width: 20, height: 3, bgcolor: "#00b894", borderRadius: 1 }} />
          <Typography variant="caption">Прогноз</Typography>
        </Box>
      </Box>
    </CardContent>
    <Divider sx={{ mb: 3 }} />
    <ResponsiveContainer width="100%" height={450}>
      <ComposedChart data={data}>
        <defs>
          <linearGradient id="colorActual" x1="0" y1="0" x2="0" y2="1">
            <stop offset="5%" stopColor="#3f51b5" stopOpacity={0.3}/>
            <stop offset="95%" stopColor="#3f51b5" stopOpacity={0}/>
          </linearGradient>
          <linearGradient id="colorPredicted" x1="0" y1="0" x2="0" y2="1">
            <stop offset="5%" stopColor="#00b894" stopOpacity={0.3}/>
            <stop offset="95%" stopColor="#00b894" stopOpacity={0}/>
          </linearGradient>
        </defs>
        <CartesianGrid strokeDasharray="3 3" stroke={alpha(theme.palette.divider, 0.5)}
vertical={false} />
        <XAxis dataKey="date" tick={{ fontSize: 12, fill: theme.palette.text.secondary }} />
        <YAxis tick={{ fontSize: 12, fill: theme.palette.text.secondary }} domain={['auto',
'auto']} />
        <Tooltip contentStyle={{ borderRadius: 12, border: "none", boxShadow: `0 8px 30px
${alpha(theme.palette.common.black, 0.12)}`, background:
alpha(theme.palette.background.paper, 0.95), backdropFilter: "blur(10px)" }} />
        <Legend iconType="circle" wrapperStyle={{ padding: 20 }} />
        <Area type="monotone" dataKey="actual" stroke="#3f51b5" strokeWidth={2}
fill="url(#colorActual)" dot={{ false }} name="Фактические данные" activeDot={{ r: 6, fill:
"#3f51b5" }} />
        <Line type="monotone" dataKey="predicted" stroke="#00b894" strokeWidth={3}
dot={{ false }} name="Прогноз" strokeDasharray="5 5" activeDot={{ r: 6, fill: "#00b894" }} />
      </ComposedChart>
    </ResponsiveContainer>
  </Card>

```

```

<Box sx={{ mt: 3, textAlign: "center" }}>
  <Typography variant="caption" color="text.secondary">
    {model2
      ? "• Прогноз построен ансамблем двух моделей • Гиперпараметры и вес можно
настроить"
      : "• Прогноз построен одной моделью • Гиперпараметры можно настроить или
использовать автоподбор"}
  </Typography>
</Box>
</Box>
<style jsx>{`
  @keyframes spin {
    from { transform: rotate(0deg); }
    to { transform: rotate(360deg); }
  }
  .spin {
    animation: spin 1s linear infinite;
  }
`}</style>
</Box>
);
}

```